Achernar

Unified Astronomy Imaging and Control System

System Architecture, AI/ML Design, and Autonomous Control Specification

Ibrahim Shahid

Lead Architect

December 19, 2025 (revised March 2026)

Abstract

Achernar is a unified astronomy imaging and control system for amateur and professional astronomers, providing a fluid offline-to-online experience. The platform centers on a lightweight cross-platform daemon written in Rust that interfaces directly with USB-connected astronomy hardware.

This document presents the full system architecture, hardware interfaces, processing pipelines, AI/ML components with reliability features, and—new in this revision—a complete specification for *closed-loop AI-driven hardware control*. Achernar's hybrid AI architecture runs quantized ONNX models at sub-50 ms latency on the edge for focus scoring, cloud detection, and direct hardware parameter control, while the cloud hosts a novel tri-modal embedding space (Image, Time-Series, Spectra) for recommendation engines and anomaly detection. The AI Parameter Control Layer formalizes the transition from AI-advisory to bounded autonomous operation via a deterministic safety supervisor, action arbiter, per-rig device profiles, and shadow-mode evaluation infrastructure.

Contents

1 Executive Summary **4**
1.1 Key Features . . . 4
1.2 Target Users . . . 4

2 System Architecture Overview **4**
2.1 Architecture Layers . . . 4

3 Equipment Layer **5**
3.1 Supported Hardware . . . 5

4 Achernar Daemon (Rust + C++ Core) **5**
4.1 Hardware Abstraction . . . 5
4.2 Image Processing Engine (C++/OpenCV + Rust) . . . 5
4.3 AI/ML Edge Processing (ONNX Runtime) . . . 5
4.4 Control Systems . . . 5
4.5 Local Storage (SQLite + Files) . . . 5
4.6 Daemon Server . . . 6

5 Interfaces, Data Contracts, and Reliability **6**
5.1 API Specifications . . . 6
5.1.1 Core Daemon Endpoints . . . 6
5.2 Core SQLite Schema . . . 6
5.3 Error Handling & Recovery . . . 7
5.4 Core Configuration . . . 7

6 AI/ML System Design **7**
6.1 Design Principles . . . 7
6.2 Live Image Intelligence (Edge, Real-Time) . . . 8
6.2.1 Sharpness / Focus Score . . . 8
6.2.2 Background/SNR Estimator . . . 8
6.2.3 Elongation / Trailing Detector . . . 8
6.2.4 Cloud/Obstruction Classifier . . . 8
6.2.5 Focus Drift Tracker . . . 8
6.3 Settings & Exposure Recommendation (Edge→Cloud, Adaptive) . . . 8
6.4 Planning & Target Ranking (Cloud) . . . 8
6.5 Post-Capture Analytics (Cloud) . . . 8
6.6 Tri-Modal Embedding (Image ↔ Light Curve ↔ Spectrum ↔ Metadata) . . . 8
6.7 Model Deployment & Lifecycle Management . . . 9

7 AI Parameter Control & Closed-Loop Hardware Autonomy **9**
7.1 Control Philosophy . . . 9
7.2 Formal Control Specification per Subsystem . . . 9
7.3 New Daemon Modules . . . 10
7.3.1 control_supervisor.rs . . . 10
7.3.2 action_arbiter.rs . . . 10
7.3.3 safety_guard.rs . . . 11
7.3.4 focus_agent.rs — State Machine . . . 11
7.3.5 focus_inference.rs — Output Types . . . 12
7.4 Database Schema Extensions . . . 12
7.4.1 rigs . . . 12

7.4.2 focuser_profiles . . . 12
7.4.3 focuser_filter_offsets . . . 13
7.4.4 focus_runs . . . 13
7.4.5 focus_samples . . . 14
7.4.6 focus_predictions . . . 15
7.4.7 focus_actions . . . 15
7.4.8 focus_drift_events . . . 16
7.4.9 focus_model_registry . . . 16
7.4.10 Recommended Indexes . . . 17
7.5 Focus Control API . . . 17
7.5.1 Error Model . . . 17
7.5.2 Control Endpoints . . . 17
7.5.3 Status & Introspection Endpoints . . . 18
7.5.4 Profile & Calibration Endpoints . . . 18
7.5.5 WebSocket Events . . . 19
7.6 ML Model Specifications . . . 19
7.6.1 Model 1: Focus Quality Estimator (FQE) . . . 19
7.6.2 Model 2: Focus Offset Predictor (FOP) . . . 21
7.6.3 Model 3: Focus Drift Forecaster (FDF) . . . 23
7.6.4 Model 4: Cloud / Obstruction Classifier (COC) . . . 24
7.6.5 Model 5: Trailing / Elongation Detector (TED) . . . 25
7.6.6 Model 6: Background / SNR Estimator . . . 26
7.6.7 Model 7: Exposure / Gain Recommender (LinUCB Contextual Bandit) . . . 26
7.6.8 Model 8: Filter Order Recommender (FOR) . . . 27
7.6.9 Model 9: Tri-Modal Embedding (TME) . . . 28
7.6.10 Edge Model Filesystem Layout and Metadata . . . 29
7.7 Training Pipeline Specification . . . 30
7.7.1 Dataset Contracts . . . 30
7.7.2 Split Strategy . . . 30
7.7.3 Preprocessing . . . 30
7.7.4 Training Stages . . . 31
7.7.5 ML Project Layout . . . 31
7.8 Focus Controller Algorithm . . . 32
7.8.1 Bounded AI Autofocus (Production MVP) . . . 32
7.8.2 Classical Fallback . . . 32
7.9 Focus Configuration . . . 32
7.10 Evaluation & Promotion Criteria . . . 33
7.10.1 Model Metrics . . . 33
7.10.2 Operational Control Metrics . . . 33
7.10.3 Testing Stages . . . 33
7.11 Autonomy Maturity Ladder & Implementation Roadmap . . . 34
7.11.1 Phase Build Order . . . 34
7.11.2 Extension Pattern to Other Subsystems . . . 34

8 Subsystem Control Architecture: Filter Wheel, Mount, and Shared Perception 34
8.1 Shared Perception Bus . . . 35
8.1.1 SharedObservation Struct . . . 35
8.1.2 Vision Layer to Subsystem Feed Table . . . 36
8.1.3 Vision Loop Runtime . . . 36
8.2 Filter Wheel Subsystem . . . 37

8.2.1 Filter-Focus Handshake Protocol . . . 37
8.2.2 Model FW-1: Per-Filter Sky Scorer (Classical) . . . 38
8.2.3 Model FW-2: Filter Sequencer (LinUCB Contextual Bandit) . . . 38
8.2.4 Model FW-3: Session Plan Optimizer (ILP, OR-Tools) . . . 39
8.2.5 Filter Wheel Safety Configuration . . . 40
8.3 Mount Subsystem . . . 40
8.3.1 Model M-1: Guiding Aggressiveness Tuner (LinUCB) . . . 40
8.3.2 Model M-2: Dither Magnitude Policy (Regression MLP) . . . 41
8.3.3 Model M-3: Plate Solve Trigger Classifier (LightGBM) . . . 42
8.3.4 Model M-4: Pointing Model (Thin-Plate Spline) . . . 43
8.3.5 Mount Safety Configuration . . . 44
8.4 Schema Additions for Filter and Mount Subsystems . . . 44
8.5 Subsystem Model Summary . . . 46

9 Data Engineering & Labeling 46
9.1 Bootstrapped Labels from Classical Pipeline . . . 46
9.2 Programmatic Weak Labels (Snorkel-Style) . . . 46
9.3 Public Archives . . . 46
9.4 Augmentations . . . 47

10 Metrics 47
10.1 Edge Models . . . 47
10.2 Recommender Systems . . . 47
10.3 System-Level Metrics . . . 47

11 Training/Serving Stack 47
11.1 Development Infrastructure . . . 47
11.2 Serving Infrastructure . . . 47

12 Edge vs Cloud: Who Runs What? 47
12.1 Edge (<50 ms Latency) . . . 47
12.2 Cloud (Heavier Compute) . . . 48

13 Network Layer 48

14 Achernar Web Platform (AWS/GCP) 48
14.1 Control & Data Planes . . . 48
14.2 Backend Services . . . 48
14.3 AI/ML Cloud Services . . . 48

15 Pragmatic Cautions 48

16 Updated R&D Roadmap 48
16.1 Weeks 1–2: Foundation . . . 48
16.2 Weeks 3–4: Quality Control + Control Foundation . . . 49
16.3 Weeks 5–6: Embeddings, Ranking, and Offset Predictor . . . 49
16.4 Weeks 7–8: Multi-Modal, Feedback, and Shadow Mode . . . 49
16.5 Weeks 9–10: Bounded Autonomy . . . 49

A Technology Stack Summary 49
A.1 Core Daemon . . . 49
A.2 Desktop Application . . . 49
A.3 Web Platform . . . 49

A.4 Key Libraries 49

B Achernar vs. Traditional Approaches **49**

C Future Research Directions **50**

D Hardware Requirements **50**

1 Executive Summary

Achernar is a unified astronomy imaging and control system designed to support both amateur and professional astronomers through a fluid offline-to-online user experience. The platform consists of a lightweight, cross-platform daemon written in Rust that interfaces directly with USB-connected peripherals such as astrophotography cameras and telescope mounts.

This daemon enables real-time frame capture, high-speed I/O, hardware-accelerated debayering and video encoding (H.264/MJPEG), and execution of compact ONNX-based AI models that dynamically score focus, suggest optimal exposure and gain, and recommend filter choices—all without requiring an internet connection.

1.1 Key Features

- **Offline-First Design:** Full functionality without internet connectivity
- **Edge AI Processing:** Real-time focus scoring, cloud detection, and imaging recommendations
- **Closed-Loop AI Control:** AI models govern focuser, exposure, and filter parameters within deterministic safety bounds
- **Hardware Acceleration:** GPU-accelerated image processing and video encoding when available
- **Multi-Platform Support:** Web interface, desktop application, and mobile PWA
- **Professional-Grade Calibration:** Automatic dark, flat, and dark-flat frame integration
- **Cloud Synchronization:** Automatic session sync when connectivity is restored
- **Uncertainty-Aware AI:** All predictions include confidence scores for reliable decision-making
- **Active Learning:** User feedback continuously improves model performance

1.2 Target Users

- **Casual Users:** Backyard astronomers using a browser-based web interface
- **Professional Astronomers:** Remote observatory operators in low-connectivity environments
- **Students & Researchers:** Educational institutions and field-deployable research setups
- **Individual Enthusiasts:** Users running the system on personal laptops

2 System Architecture Overview

The Achernar system is designed with a layered architecture that separates concerns between hardware interfacing, data processing, AI/ML inference, user interfaces, and cloud services. This design enables both edge computing for real-time operations and cloud computing for collaboration, archiving, and large-scale model training.

2.1 Architecture Layers

1. **Equipment Layer:** Physical hardware devices (cameras, mounts, focusers, filter wheels, sensors)
2. **Daemon Layer:** Rust-based core with hardware abstraction, image processing, AI inference, and the AI Control Layer
3. **Network Layer:** Local WiFi AP, internet connectivity, and VPN tunneling
4. **Client Layer:** Desktop applications, web browsers, tablets, and mobile devices
5. **Cloud Platform:** Django-based web services with AI/ML infrastructure
6. **External Services:** Weather APIs, astronomical catalogs, and astrometry services

3 Equipment Layer

3.1 Supported Hardware

- **Cameras:** ZWO ASI SDK, ToupTek SDK, libgphoto2 (DSLR support)
- **Telescope Mounts:** INDI Client (libindi), ASCOM Alpaca, Serial/TCP protocols
- **Focusers:** INDI (libindi-dev), Moonlite protocol, USB/Serial interfaces
- **Filter Wheels:** INDI drivers, USB HID, serial commands
- **Environmental Sensors:** Arduino/USB, OneWire DS18B20, BME280 (I2C)

4 Achernar Daemon (Rust + C++ Core)

The daemon is the heart of the Achernar system, providing hardware abstraction, real-time image processing, AI inference, and—with the AI Control Layer—closed-loop parameter control.

4.1 Hardware Abstraction

- USB API: libusb 1.0, tokio-serial, hidapi-rs
- INDI Layer: INDI client (Rust), libindi bindings, TCP async
- Camera Drivers: Camera SDK bindings, FFI to C++, USB async I/O

4.2 Image Processing Engine (C++/OpenCV + Rust)

- **Raw Buffer:** `cv::Mat` + `ndarray`, FITS (`fitsio-rs`), TIFF (`image-rs`), shared memory
- **Calibration:** OpenCV algorithms, dark/flat processing, Rust orchestration, libraw bindings
- **Alignment:** Feature detection (AKAZE/ORB), opencv-rust, RANSAC matching
- **Stacker:** C++ core, Rust threading, tokio async, memory-mapped operations
- **Post-Processing:** Histogram stretch, C++ processing, Rust coordination, GPU acceleration

4.3 AI/ML Edge Processing (ONNX Runtime)

Compact AI models run locally for real-time guidance with uncertainty quantification and explainability. With the AI Control Layer described in Section 7, these models now also drive hardware parameters directly within deterministic safety bounds.

4.4 Control Systems

- **Sequence Controller:** State machine, JSON configs, async/await, event-driven
- **AI Recommender:** Contextual bandit (LinUCB/Thompson), Bayesian updates
- **Auto Focus:** HFD/HFR calculation, curve fitting, AI-guided step optimization
- **Plate Solver:** Local solver with index files, FITS WCS, star detection
- **AI Control Layer** (new): Closed-loop parameter control with safety supervision — see Section 7

4.5 Local Storage (SQLite + Files)

- SQLite Database: 3.40+, WAL mode, encrypted DB, schema migrations
- Raw Files: FITS format, memory-mapped, SSD storage, LZ4 compression
- Stacked Images: 16/32-bit TIFF, FITS with WCS, metadata embedded, thumbnail cache
- Logs: Structured logging (tracing-rs), JSON format, rotation policy
- Configuration: TOML configs, hot reloading, equipment profiles, Git-tracked

- ML Cache: Model artifacts, ONNX files, embeddings, feature store

4.6 Daemon Server

Axum web server, WebSocket (tokio-tungstenite), gRPC (tonic), JSON/MessagePack serialization.

5 Interfaces, Data Contracts, and Reliability

5.1 API Specifications

Achernar uses REST + WebSocket telemetry with JSON payloads. The daemon is the local control plane; the cloud is the collaboration/archival plane.

REST: `http://localhost:3030/api/v1` **WebSocket:** `ws://localhost:3030/ws/telemetry`

5.1.1 Core Daemon Endpoints

Method	Endpoint	Description	Notes
GET	`/status`	Global equipment state	JSON
POST	`/camera/capture`	Trigger image/sequence	JSON
POST	`/mount/slew`	Slew/GoTo coordinates	JSON
GET	`/stream/mjpeg`	Live view stream	`?fps=10&scale=0.5`
POST	`/ai/analyze`	Manual AI frame analysis	JSON

Focus-specific control endpoints are specified in full in Section 7.5.

5.2 Core SQLite Schema

```
CREATE TABLE sessions (
    id          TEXT PRIMARY KEY,
    start_time  DATETIME NOT NULL,
    target_name TEXT,
    site_lat    REAL,
    site_lon    REAL,
    sync_status TEXT DEFAULT 'pending'
);

CREATE TABLE frames (
    id               TEXT PRIMARY KEY,
    session_id       TEXT REFERENCES sessions(id),
    filename         TEXT NOT NULL,
    filter           TEXT,
    exposure_seconds REAL,
    temperature      REAL,
    focus_hfr        REAL,
    cloud_prob       REAL,
    background_adu   REAL,
    is_rejected      BOOLEAN DEFAULT 0,
    created_at       DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE telemetry_log (
    timestamp   DATETIME,
    metric_name TEXT,
    value       REAL
);
```

Listing 1: sessions and frames tables

Extended focuser schema is in Section 7.4.

5.3 Error Handling & Recovery

Failure	Detection	Recovery	Notification
Camera Disconnect	USB I/O error/-timeout	Pause; reset USB; retry (3×)	"Camera lost. Retrying..."
Plate Solve Fail	Timeout / NULL solve	Spiral search; increase exposure	"Solve failed. Searching..."
Cloud Detected	cloud_prob > 0.8	Pause imaging; park mount	"Clouds detected. Paused."
Guiding Lost	RMS > 2.0 for 10 s	Discard; recalibrate	"Guiding drift. Recalibrating."

5.4 Core Configuration

Listing 2: achernar.toml — core blocks

```
[server]
host      = "0.0.0.0"
port      = 3030
log_level = "info"

[equipment.camera]
driver       = "zwo_asi"
id           = "ASI533MC_Pro"
gain_default = 100
offset       = 50

[equipment.mount]
driver      = "indi"
device_name = "iOptron HEM27"
host        = "localhost"
port        = 7624

[ai]
enable_edge_inference = true
model_path            = "./models/onnx"
cloud_threshold       = 0.75
```

6 AI/ML System Design

6.1 Design Principles

1. **Scientific Integrity First:** Never alter raw science data with ML
2. **Uncertainty-Aware:** All predictions include confidence scores
3. **Explainable Decisions:** Critical recommendations show reasoning
4. **Graceful Degradation:** System works without AI; AI enhances, does not replace classical methods
5. **Continuous Learning:** User feedback loops back to improve models
6. **Privacy-Preserving:** Support federated learning without data upload

6.2 Live Image Intelligence (Edge, Real-Time)

6.2.1 Sharpness / Focus Score

Goal: Predict focus quality at 1–5 Hz with confidence intervals. Input: downscaled mono frame (256–384 px), optional star cutouts. Model: ViT-Tiny or MobileNetV3-Small (quantized INT8), ONNX Runtime. Output: HFR prediction + uncertainty (MC Dropout or ensemble variance). Labels: HFR/HFD from classical star fitting (Photutils/SEP). Loss: MAE + pairwise ranking loss + calibration loss (Platt scaling). Extras: temperature/airmass features fused via small MLP.

6.2.2 Background/SNR Estimator

Fast classical estimation: sigma-clipped background, biweight midvariance, star masking via SEP. Optional ML: tiny DnCNN-lite noise map. Uncertainty via bootstrap resampling.

6.2.3 Elongation / Trailing Detector

Star ellipse fitting: ellipticity $e = 1 - b/a$, histogram + Rayleigh test on position angles. Alert threshold: only trigger when confidence > 0.85.

6.2.4 Cloud/Obstruction Classifier

MobileNetV3-Small binary classifier. Hard negative mining. Temperature scaling for calibration. Fallback rule: sudden star count drop + background rise + humidity change.

6.2.5 Focus Drift Tracker

Kalman filter on focus metric + ARIMA/Prophet or linear trend with temperature. Forecast drift 10–30 minutes ahead with 95% confidence intervals. Adaptive per-equipment drift rates.

6.3 Settings & Exposure Recommendation (Edge→Cloud, Adaptive)

v0 Rules: Target surface brightness, moon separation, filter, pixel scale, sky SQM.

v1 Adaptive: Contextual bandit (LinUCB / Thompson Sampling). Context $\mathbf{x}$: [filter, SB, airmass, moon alt, moon sep, SQM, seeing, pixel scale, sensor params]. Reward r: sharpness↑, SNR↑, saturation↓.

v2 Global: Bayesian hierarchical regression with optional federated learning.

6.4 Planning & Target Ranking (Cloud)

Observability score f(altitude, airmass, moon sep, meridian flip time, seeing, clouds). Weighted interval scheduling / ILP (OR-Tools). FAISS (IVF-PQ) nearest-neighbor on image embeddings for framing suggestions.

6.5 Post-Capture Analytics (Cloud)

Calibrated logistic regression on engineered features for quality scoring. Isolation Forest / One-Class SVM for anomaly detection. Live stacking IQ weighting by $1/\text{FWHM}^2$.

6.6 Tri-Modal Embedding (Image ↔ Light Curve ↔ Spectrum ↔ Metadata)

- **Image Encoder:** ViT-Small or ConvNeXt-Tiny (ImageNet pretrained, astro fine-tuned)
- **Light Curve:** Temporal Transformer or InceptionTime on (time, mag, err)

- **Spectrum:** 1D Transformer or 1D ResNet after continuum normalization
- **Metadata:** Small Transformer/MLP on tabular features
- **Alignment:** Pairwise InfoNCE / CLIP-style; symmetric cross-entropy for I↔L, I↔S, L↔S; $d = 256$ shared space
- **Vector DB:** FAISS IVF-PQ

6.7 Model Deployment & Lifecycle Management

Semantic versioning (major.minor.patch). Canary deployments (5% → 25% → 100%). Shadow mode: new models run alongside production, predictions logged but not acted upon. Monitoring: inference latency (p50/p95/p99), accuracy on holdout sets, feature drift (KL divergence).

7 AI Parameter Control & Closed-Loop Hardware Autonomy

Design Note

This section represents the evolution of Achernar from an *AI-enhanced* observation system to a *closed-loop autonomous control system*. Every controllable subsystem is formalized as: observe → estimate state → choose action → enforce safety → actuate → verify outcome.

7.1 Control Philosophy

The key shift is that AI models no longer only score images or recommend parameters—they *drive hardware parameters in real time* within a stack of safety guarantees. Two paradigms are distinguished:

AI-Assisted Control (primary production mode). AI infers state and proposes parameters; a deterministic controller executes them within auditable safety bounds. Predictable, debuggable, hardware-safe.

End-to-End AI Control (research mode, deferred). AI directly outputs hardware commands. Dangerous on expensive hardware without extensive simulation-first training. Deferred until a simulation environment and offline replay harness are mature.

Important

The correct development order is: (1) classical baseline, (2) AI perception, (3) AI proposal, (4) AI parameter tuning in bounded mode, (5) optional RL in simulation only. Do not start with RL on physical hardware.

7.2 Formal Control Specification per Subsystem

Each controllable subsystem is treated as a constrained controller:

- o_t — **Observation**: frame, focuser position, temperature, filter, history
- s_t — **Latent state**: true optical focus error, backlash state, thermal drift trend
- a_t — **Action**: move focuser $\pm n$ steps, change exposure, switch filter, pause imaging
- C/R — **Cost/Reward**: lower HFR better, fewer moves better, hitting hard stop catastrophic
- **Constraints**: travel range, ownership locks, confidence thresholds, settle timing

7.3 New Daemon Modules

The following Rust modules are added inside the daemon:

```
crates/achernar-daemon/src/
  control/
    mod.rs
    control_supervisor.rs   // autonomy mode management
    action_arbiter.rs       // priority-based conflict resolution
    safety_guard.rs         // hardware firewall (AI -> driver)
    ownership.rs            // device lock tracking
  focus/
    mod.rs
    focus_agent.rs          // run orchestrator / state machine
    classical_focus_solver.rs
    focus_policy.rs         // deterministic policy layer
    focus_inference.rs      // ONNX model runner
    focus_features.rs       // sliding feature window
    focus_drift.rs          // proactive refocus forecaster
    backlash.rs             // backlash compensation
  storage/
    focus_runs_repo.rs
    control_actions_repo.rs
    device_profiles_repo.rs
  models/
    onnx_runner.rs
    model_registry.rs
```

Listing 3: New control and focus module layout

7.3.1 control_supervisor.rs

Decides the autonomy mode for each subsystem and enforces mode downgrades on repeated faults.

```
pub enum AutonomyMode {
    Manual,
    Advisory,
    SupervisedAuto,
    FullAuto,
    FallbackOnly,
}
```

7.3.2 action_arbiter.rs

Resolves priority and device ownership when multiple subsystems want to act simultaneously. Priority order (highest to lowest): emergency stop > collision/safety > mount recovery > autofocus > exposure tuning > convenience recommendations.

```
pub enum ControlDomain { Focus, Camera, Mount, FilterWheel, Sequence,
    Safety }

pub struct ActionRequest {
    pub domain:   ControlDomain,
    pub priority: u8,
    pub owner:    String,
    pub action:   ControlAction,
}
```

7.3.3 safety_guard.rs

The most critical new module. Sits directly between AI output and hardware commands. All AI outputs must pass through this module before touching hardware.

```
pub struct FocusSafetyConfig {
    pub min_position:             i32,
    pub max_position:             i32,
    pub max_step_single:          i32,
    pub max_total_travel_per_run: i32,
    pub min_confidence_to_act:    f32,
    pub max_failed_improvements:  u8,
    pub settle_ms:                u64,
    pub backlash_steps:           i32,
}
```

7.3.4 focus_agent.rs — State Machine

```
pub enum FocusRunState {
    Idle, Initializing, CapturingBaseline, PredictingOffset,
    Moving, Settling, Measuring, Evaluating, Converged,
    FallbackSweep, Failed, Aborted,
}
```

```
pub async fn run_autofocus(ctx: &mut FocusContext)
    -> Result<FocusOutcome, FocusError>
{
    let _run_id  = ctx.start_run().await?;
    let baseline = ctx.capture_and_measure().await?;
    ctx.log_sample(&baseline).await?;

    if !baseline.observation_ok() {
        return ctx.run_classical_fallback("poor_baseline").await;
    }
    let offset = ctx.infer_focus_offset(&baseline).await?;
    ctx.log_inference(&offset).await?;

    if offset.uncertainty_steps > ctx.cfg.max_uncertainty_steps {
        return ctx.run_classical_fallback("uncertainty_too_high").
   await;
    }

    let mut attempts = 0;
    let mut current  = baseline;
    loop {
        let proposed  = ctx.policy.propose_action(&current, &offset)
   ?;
        let validated = ctx.safety_guard.validate_focus_action(
            proposed, ctx.current_position())?;

        ctx.execute_focus_action(&validated).await?;
        ctx.log_action(&validated).await?;
        ctx.settle().await?;

        let next = ctx.capture_and_measure().await?;
        ctx.log_sample(&next).await?;

        if ctx.policy.is_converged(&current, &next) {
```

```
            return ctx.finish_success(next).await;
        }
        if !ctx.policy.improved(&current, &next) { attempts += 1; }
        if attempts > ctx.cfg.max_failed_improvements {
            return ctx.run_classical_fallback("no_improvement").await
    ;
        }
        current = next;
    }
}
```

Listing 4: Focus agent main loop (pseudocode)

7.3.5 focus_inference.rs — Output Types

```
pub struct FocusQualityOutput {
    pub predicted_hfr:   f32,
    pub sharpness_score: f32,
    pub uncertainty:     f32,
}
pub struct FocusOffsetOutput {
    pub delta_steps:              f32,
    pub uncertainty_steps:        f32,
    pub recommended_search_radius: i32,
}
```

7.4 Database Schema Extensions

Recommended SQLite settings for control workloads:

```
PRAGMA foreign_keys = ON;
PRAGMA journal_mode  = WAL;
PRAGMA synchronous   = NORMAL;
PRAGMA temp_store    = MEMORY;
PRAGMA busy_timeout  = 5000;
```

7.4.1 rigs

```
CREATE TABLE rigs (
    id                        TEXT PRIMARY KEY,
    name                      TEXT NOT NULL,
    telescope_model           TEXT,
    telescope_aperture_mm     REAL,
    telescope_focal_length_mm REAL,
    focuser_model             TEXT,
    camera_model              TEXT,
    filter_wheel_model        TEXT,
    reducer_model             TEXT,
    pixel_size_um             REAL,
    notes                     TEXT,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

7.4.2 focuser_profiles

```
CREATE TABLE focuser_profiles (
    rig_id TEXT PRIMARY KEY REFERENCES rigs(id) ON DELETE CASCADE,

    min_position  INTEGER NOT NULL,
    max_position  INTEGER NOT NULL,
    home_position INTEGER,
    park_position INTEGER,

    backlash_steps               INTEGER NOT NULL DEFAULT 0,
    preferred_approach_direction TEXT    NOT NULL DEFAULT 'inward'
        CHECK (preferred_approach_direction IN ('inward','outward','none')),

    max_step_single              INTEGER NOT NULL DEFAULT 100,
    max_total_travel_per_run     INTEGER NOT NULL DEFAULT 2000,
    default_settle_ms            INTEGER NOT NULL DEFAULT 500,
    min_star_count               INTEGER NOT NULL DEFAULT 20,

    critical_focus_zone_steps    REAL,
    temp_coefficient_steps_per_c REAL,
    filter_offset_source         TEXT NOT NULL DEFAULT 'manual'
        CHECK (filter_offset_source IN ('manual','measured','learned')),

    autofocus_enabled  INTEGER NOT NULL DEFAULT 1,
    ai_control_enabled INTEGER NOT NULL DEFAULT 0,

    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

7.4.3 focuser_filter_offsets

```
CREATE TABLE focuser_filter_offsets (
    rig_id       TEXT NOT NULL REFERENCES rigs(id) ON DELETE CASCADE,
    filter_name  TEXT NOT NULL,
    offset_steps INTEGER NOT NULL DEFAULT 0,
    offset_uncertainty_steps REAL,
    source       TEXT NOT NULL DEFAULT 'manual'
        CHECK (source IN ('manual','measured','learned')),
    sample_count     INTEGER NOT NULL DEFAULT 0,
    last_measured_at DATETIME,
    PRIMARY KEY (rig_id, filter_name)
);
```

7.4.4 focus_runs

```
CREATE TABLE focus_runs (
    id         TEXT PRIMARY KEY,
    session_id TEXT REFERENCES sessions(id) ON DELETE SET NULL,
    rig_id     TEXT NOT NULL REFERENCES rigs(id) ON DELETE CASCADE,

    trigger_type TEXT NOT NULL CHECK (trigger_type IN (
        'manual','scheduled','temp_drift','filter_change',
        'startup','post_meridian_flip','quality_drop','recovery')),

    autonomy_mode TEXT NOT NULL CHECK (autonomy_mode IN (
        'manual','advisory','supervised_auto',
        'full_auto','fallback_only','shadow')),

    controller_type TEXT NOT NULL
        CHECK (controller_type IN ('classical','hybrid_ai','ai_bounded')),
```

```
    state TEXT NOT NULL CHECK (state IN (
        'initializing','capturing_baseline','predicting_offset',
        'moving','settling','measuring','refining',
        'converged','fallback','failed','aborted')),

    filter_name         TEXT,
    preview_exposure_ms INTEGER NOT NULL,
    roi_mode            TEXT NOT NULL DEFAULT 'center'
        CHECK (roi_mode IN ('full_frame','center','star_patch','multi_patch')
    ),

    start_position    INTEGER NOT NULL,
    final_position    INTEGER,
    best_fit_position INTEGER,

    initial_hfr REAL, final_hfr REAL, best_hfr REAL,

    total_moves         INTEGER NOT NULL DEFAULT 0,
    total_travel_steps  INTEGER NOT NULL DEFAULT 0,
    settle_time_ms_avg  REAL,
    fallback_used       INTEGER NOT NULL DEFAULT 0,
    success             INTEGER NOT NULL DEFAULT 0,

    failure_reason TEXT,
    aborted_by     TEXT,

    temp_c              REAL,
    humidity_pct        REAL,
    sky_background_adu  REAL,
    star_count_baseline INTEGER,

    model_quality_version TEXT,
    model_offset_version  TEXT,
    model_policy_version  TEXT,

    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    started_at DATETIME NOT NULL,
    ended_at   DATETIME
);
```

7.4.5 focus_samples

```
CREATE TABLE focus_samples (
    id           TEXT PRIMARY KEY,
    focus_run_id TEXT NOT NULL REFERENCES focus_runs(id) ON DELETE CASCADE,

    sample_index       INTEGER  NOT NULL,
    timestamp          DATETIME NOT NULL,
    focuser_position   INTEGER  NOT NULL,
    movement_direction TEXT CHECK (movement_direction IN ('inward','outward
    ','none')),

    frame_id       TEXT REFERENCES frames(id) ON DELETE SET NULL,
    frame_path     TEXT,
    thumbnail_path TEXT,
    crop_path      TEXT,

    filter_name         TEXT,
    preview_exposure_ms INTEGER NOT NULL,
    gain                INTEGER,
    sensor_temp_c       REAL,
    ambient_temp_c      REAL,
```

```
    humidity_pct        REAL,

    star_count          INTEGER,
    star_count_used     INTEGER,
    saturation_fraction REAL,
    background_adu      REAL,
    background_sigma    REAL,

    classical_hfr         REAL,
    classical_hfd         REAL,
    classical_fwhm        REAL,
    classical_ellipticity REAL,
    fit_quality           REAL,
    measurement_valid     INTEGER NOT NULL DEFAULT 1,

    sharpness_score      REAL,
    predicted_hfr        REAL,
    quality_uncertainty  REAL,

    focus_offset_pred_steps  REAL,
    offset_uncertainty_steps REAL,

    notes TEXT,
    UNIQUE (focus_run_id, sample_index)
);
```

7.4.6 focus_predictions

```
CREATE TABLE focus_predictions (
    id              TEXT PRIMARY KEY,
    focus_run_id    TEXT REFERENCES focus_runs(id)    ON DELETE CASCADE,
    focus_sample_id TEXT REFERENCES focus_samples(id) ON DELETE CASCADE,

    prediction_type TEXT NOT NULL
        CHECK (prediction_type IN ('quality','offset','drift','policy')),

    model_name    TEXT NOT NULL,
    model_version TEXT NOT NULL,
    inference_mode TEXT NOT NULL
        CHECK (inference_mode IN ('live','shadow','replay','offline_rescore')
    ),

    input_schema_version TEXT,
    output_json          TEXT NOT NULL,
    confidence           REAL,
    latency_ms           REAL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

7.4.7 focus_actions

```
CREATE TABLE focus_actions (
    id             TEXT PRIMARY KEY,
    focus_run_id   TEXT NOT NULL REFERENCES focus_runs(id) ON DELETE CASCADE,
    before_sample_id TEXT REFERENCES focus_samples(id) ON DELETE SET NULL,
    after_sample_id  TEXT REFERENCES focus_samples(id) ON DELETE SET NULL,

    action_index INTEGER  NOT NULL,
    timestamp    DATETIME NOT NULL,

    proposed_direction    TEXT CHECK (proposed_direction   IN ('inward','
    outward','stop','fallback')),
```

```
    proposed_delta_steps  INTEGER,
    proposed_reason       TEXT,
    validated_direction   TEXT CHECK (validated_direction  IN ('inward','
    outward','stop','fallback')),
    validated_delta_steps INTEGER,
    executed_direction    TEXT CHECK (executed_direction   IN ('inward','
    outward','stop','fallback')),
    executed_delta_steps  INTEGER,

    was_clamped            INTEGER NOT NULL DEFAULT 0,
    was_rejected           INTEGER NOT NULL DEFAULT 0,
    rejection_reason       TEXT,
    backlash_applied_steps INTEGER NOT NULL DEFAULT 0,

    expected_improvement REAL,
    observed_hfr_before  REAL,
    observed_hfr_after   REAL,
    observed_improvement REAL,

    confidence    REAL,
    policy_source TEXT NOT NULL
        CHECK (policy_source IN ('classical','hybrid_ai','safety_override','
    manual')),

    UNIQUE (focus_run_id, action_index)
);
```

7.4.8 focus_drift_events

```
CREATE TABLE focus_drift_events (
    id           TEXT PRIMARY KEY,
    rig_id       TEXT NOT NULL REFERENCES rigs(id) ON DELETE CASCADE,
    focus_run_id TEXT REFERENCES focus_runs(id) ON DELETE SET NULL,

    current_filter_name  TEXT,
    last_focus_position  INTEGER NOT NULL,
    current_position     INTEGER,
    temp_at_last_focus_c REAL,
    current_temp_c       REAL,
    elapsed_minutes      REAL,

    predicted_drift_steps REAL NOT NULL,
    uncertainty_steps     REAL,
    recommended_refocus   INTEGER NOT NULL DEFAULT 0,
    recommended_by_at     DATETIME,
    actual_refocus_run_id TEXT REFERENCES focus_runs(id) ON DELETE SET NULL,

    model_version TEXT,
    created_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

7.4.9 focus_model_registry

```
CREATE TABLE focus_model_registry (
    model_name    TEXT NOT NULL,
    model_version TEXT NOT NULL,
    model_type    TEXT NOT NULL
        CHECK (model_type IN ('quality','offset','drift','policy')),

    file_path             TEXT NOT NULL,
    sha256                TEXT NOT NULL,
```

```
    input_schema_version  TEXT NOT NULL,
    output_schema_version TEXT NOT NULL,
    quantized             INTEGER NOT NULL DEFAULT 0,
    active                INTEGER NOT NULL DEFAULT 0,

    trained_dataset_id       TEXT,
    calibration_metrics_json TEXT,
    deployment_notes         TEXT,

    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (model_name, model_version)
);
```

7.4.10 Recommended Indexes

```
CREATE INDEX idx_focus_runs_rig_started
    ON focus_runs (rig_id, started_at DESC);
CREATE INDEX idx_focus_runs_session
    ON focus_runs (session_id, started_at DESC);
CREATE INDEX idx_focus_samples_run_index
    ON focus_samples (focus_run_id, sample_index);
CREATE INDEX idx_focus_actions_run_index
    ON focus_actions (focus_run_id, action_index);
CREATE INDEX idx_focus_predictions_sample
    ON focus_predictions (focus_sample_id, prediction_type);
CREATE INDEX idx_focus_drift_rig_created
    ON focus_drift_events (rig_id, created_at DESC);
```

7.5 Focus Control API

All endpoints are prefixed with `/api/v1/focus`.

7.5.1 Error Model

```
{
  "error": {
    "code":    "FOCUS_OUT_OF_RANGE",
    "message": "Requested position exceeds focuser profile bounds.",
    "details": { "requested_position": 32880, "max_position": 32000 }
  }
}
```

Error codes: `FOCUS_BUSY`, `FOCUS_NOT_ENABLED`, `FOCUS_PROFILE_MISSING`, `FOCUS_OUT_OF_RANGE`, `FOCUS_SAFETY_REJECTED`, `FOCUS_LOW_CONFIDENCE`, `FOCUS_MEASUREMENT_INVALID`, `FOCUS_ABORTED`, `FOCUS_TIMEOUT`, `FOCUS_CALIBRATION_RUNNING`.

7.5.2 Control Endpoints

Method	Endpoint	Description
POST	`/run`	Start an autofocus run
POST	`/abort`	Abort active run
POST	`/nudge`	Bounded manual move
POST	`/move-to`	Absolute position move
POST	`/replay-evaluate`	Shadow-replay a historical run

Listing 5: POST /run — request

```
{
```

```
  "trigger_type":        "manual",
  "autonomy_mode":       "supervised_auto",
  "controller_type":     "hybrid_ai",
  "filter_name":         "Ha",
  "preview_exposure_ms": 1000,
  "roi_mode":            "center",
  "max_duration_sec":    90,
  "allow_fallback":      true
}
```

7.5.3 Status & Introspection Endpoints

Method	Endpoint	Description
GET	`/status`	Active run state + live metrics
GET	`/runs`	List recent runs
GET	`/runs/{run_id}`	Full run detail (samples, actions, predictions)
GET	`/active-run`	Shortcut to current active run
GET	`/last-best-position`	Most recent best focus by filter

Listing 6: GET /status — response

```
{
  "enabled":           true,
  "active_run_id":     "focusrun_01JXYZ...",
  "state":             "refining",
  "autonomy_mode":     "supervised_auto",
  "current_position":  18285,
  "last_sample": {
    "sample_index":       5,
    "classical_hfr":      2.17,
    "predicted_hfr":      2.14,
    "star_count":         84,
    "measurement_valid":  true
  },
  "last_prediction": {
    "focus_offset_pred_steps":  -12.6,
    "offset_uncertainty_steps":  5.8,
    "confidence":                0.88
  },
  "safety": {
    "last_action_clamped":   true,
    "last_rejection_reason": null
  }
}
```

7.5.4 Profile & Calibration Endpoints

Method	Endpoint	Description
GET	`/profile`	Retrieve safety profile + filter offsets
PUT	`/profile`	Update profile parameters
POST	`/calibrations/run`	Start a calibration run
GET	`/calibrations/{id}`	Inspect calibration result
POST	`/datasets/export`	Queue a training dataset export
GET	`/datasets/exports/{id}`	Check export status and path

7.5.5 WebSocket Events

Event	Description
FOCUS_RUN_STARTED	Run initiated
FOCUS_RUN_STATE_CHANGED	State machine transition
FOCUS_SAMPLE_CAPTURED	New measurement point logged
FOCUS_PREDICTION_MADE	Model inference complete
FOCUS_ACTION_PROPOSED	AI proposed a focuser move
FOCUS_ACTION_VALIDATED	Safety guard result (may include clamp)
FOCUS_ACTION_EXECUTED	Motor command sent
FOCUS_ACTION_REJECTED	Safety guard rejected the action
FOCUS_FALLBACK_TRIGGERED	Classical sweep activated
FOCUS_RUN_CONVERGED	Successful completion
FOCUS_RUN_FAILED	Failed with reason code
FOCUS_DRIFT_ALERT	Proactive refocus recommendation

Listing 7: FOCUS_ACTION_VALIDATED event

```
{
  "type":      "FOCUS_ACTION_VALIDATED",
  "timestamp": "2026-03-16T21:17:04Z",
  "payload": {
    "run_id":                "focusrun_01JXYZ...",
    "action_index":          3,
    "proposed_delta_steps":  -22,
    "validated_delta_steps": -12,
    "was_clamped":           true,
    "confidence":            0.86
  }
}
```

7.6 ML Model Specifications

This section specifies every model in the Achernar ML stack: architecture layer-by-layer, loss formulations, training hyperparameters, feature engineering, quantization strategy, and ONNX export constraints.

7.6.1 Model 1: Focus Quality Estimator (FQE)

Purpose. Estimate focus quality at 1–5 Hz during both active autofocus runs and continuous imaging. Provides the state estimation signal used by all downstream control models.

Architecture: Modified MobileNetV3-Small. The input stem accepts a single grayscale channel by replacing the default three-channel first convolution:

```
import torch, torch.nn as nn
from torchvision.models import mobilenet_v3_small

class FocusQualityEstimator(nn.Module):
    def __init__(self, n_meta: int = 4):
        super().__init__()
        backbone = mobilenet_v3_small(weights="IMAGENET1K_V1")
        # Adapt first conv: 3-ch -> 1-ch grayscale, average weights
        old_w = backbone.features[0][0].weight.data  # (16,3,3,3)
        new_conv = nn.Conv2d(1, 16, 3, stride=2, padding=1, bias=
   False)
        new_conv.weight.data = old_w.mean(dim=1, keepdim=True)
        backbone.features[0][0] = new_conv
```

```
        # Feature extractor up to global pool -> 576-dim
        self.features  = backbone.features   # output: (B, 576, 1, 1)
        self.pool      = backbone.avgpool
        # Optional metadata MLP: [temp_norm, filter_id, pos_norm,
    star_norm]
        self.meta_mlp  = nn.Sequential(
            nn.Linear(n_meta, 32), nn.GELU(), nn.Linear(32, 32))
        # Regression head: HFR + log-variance (heteroscedastic)
        self.head = nn.Sequential(
            nn.Linear(576 + 32, 128), nn.GELU(),
            nn.Dropout(0.20),
            nn.Linear(128, 2))     # [hfr_pred, log_sigma_sq]

     def forward(self, img, meta):
        x = self.pool(self.features(img)).flatten(1)  # (B, 576)
        m = self.meta_mlp(meta)                        # (B, 32)
        return self.head(torch.cat([x, m], dim=1))     # (B, 2)
```

Listing 8: PyTorch – FQE model definition

Layer breakdown.

Stage	Block	Output shape	Notes
Stem	Conv2d(1,16,3,s=2)+BN+Hardswish	$(B, 16, 128, 128)$	grayscale-adapted
Bottleneck 1	IR block, exp=16, out=16	$(B, 16, 64, 64)$	SE, ReLU
Bottleneck 2	IR block, exp=72, out=24	$(B, 24, 32, 32)$	SE, ReLU
Bottleneck 3	IR block, exp=88, out=24	$(B, 24, 32, 32)$	SE, ReLU
Bottleneck 4	IR block, exp=96, out=40	$(B, 40, 16, 16)$	SE, Hardswish
Bottlenecks 5-8	exp=240/120/144/288, out=40-96	$(B, 96, 8, 8)$	SE, Hardswish
Bottleneck 9-11	exp=576, out=96-96	$(B, 96, 4, 4)$	SE, Hardswish
Head conv	Conv2d(96,576,1)+BN+Hardswish	$(B, 576, 4, 4)$	channel expand
Global pool	AdaptiveAvgPool2d(1,1)	$(B, 576)$	flatten
Metadata MLP	Linear(4,32)+GELU+Linear(32,32)	$(B, 32)$	fused
Concat	—	$(B, 608)$	—
Regression head	Linear(608,128)+GELU+Drop+Linear(128,2)	$(B, 2)$	[hfr, $\log \sigma^2$]

Heteroscedastic loss formulation. Let $\hat{y}$ be the predicted HFR and $\hat{s} = \log \sigma^2$ the predicted log-variance.

$$\mathcal{L}_{\text{hetero}} = \frac{1}{N} \sum_{i=1}^{N} \left[\frac{1}{2} e^{-\hat{s}_i} \, \mathcal{H}_\delta(\hat{y}_i, y_i) + \frac{1}{2} \hat{s}_i \right] \tag{1}$$

where $\mathcal{H}_\delta$ is the Huber loss with $\delta = 0.5$ arc-sec. The $e^{-\hat{s}}$ term down-weights high-uncertainty predictions; the $\hat{s}$ penalty prevents trivial collapse to infinite variance.

Training hyperparameters.

Param	Value
Optimizer	AdamW, lr = 1e-3, weight_decay = 1e-4
Scheduler	CosineAnnealingLR, T_max = 50, $\eta_{\min}$ = 1e-6
Epochs	50 total; QAT enabled for last 5 epochs
Batch size	64, stratified by HFR quartile and filter
Precision	AMP float16 during training; INT8 export
Warmup	5 epochs linear warmup
Early stopping	Patience = 10 epochs on val MAE

Preprocessing pipeline (must match edge).

1. Extract 256×256 center crop (or top-4-star patches tiled to 256×256)
2. Convert to `float32`
3. Robust normalization: clip to [1st, 99th] percentile of pixel values, then scale to $[0, 1]$
4. Store p_1, p_{99} per-rig in `device_profiles` for identical edge preprocessing
5. Metadata vector: $[\tilde{T}, \text{filter_id}/6, \tilde{p}, \log(\tilde{n}+1)]$ (all $\in [0,1]$)

Edge quantization and ONNX export.

```
from torch.quantization import get_default_qat_qconfig, prepare_qat,
    convert

model.qconfig = get_default_qat_qconfig('fbgemm')   # x86; use '
    qnnpack' for ARM
prepare_qat(model, inplace=True)
# ... fine-tune 5 epochs with QAT enabled ...
model.eval()
model_int8 = convert(model, inplace=False)

dummy_img  = torch.zeros(1, 1, 256, 256)
dummy_meta = torch.zeros(1, 4)
torch.onnx.export(
    model_int8, (dummy_img, dummy_meta),
    "focus_quality_1.0.0.onnx",
    input_names=["image", "metadata"],
    output_names=["hfr_pred", "log_sigma_sq"],
    dynamic_axes={"image": {0: "batch"}, "metadata": {0: "batch"}},
    opset_version=17,
)
```

Listing 9: QAT export to INT8 ONNX

Latency target: < 10 ms (p50), < 20 ms (p95) on Raspberry Pi 4 (CPU, INT8 ONNX Runtime). Verified via `onnxruntime.InferenceSession` benchmark before promotion.

7.6.2 Model 2: Focus Offset Predictor (FOP)

Purpose. Estimate the signed number of focuser steps between the current position and best focus. This model directly initializes the focuser's search position, reducing move count.

Feature set (38 features).

Group	Features	Norm
Quality (3)	`predicted_hfr`, `sharpness_score`, `quality_uncertainty`	z-score
Classical (4)	`classical_hfr`, `classical_fwhm`, `fit_quality`, `star_count`	z-score / log
Position (5)	`pos_norm` ($p/p_{\max}$), `dist_from_start_norm`, `sample_idx_norm`, `dir_encoded` ($\{-1, 0, +1\}$), `run_initial_hfr`	as defined
Temperature (4)	`ambient_temp_norm`, `sensor_temp_norm`, `temp_delta_from_run_start`, `temp_coeff_norm`	z-score
Filter (7)	One-hot: L, R, G, B, Ha, OIII, SII	—
History (6)	`prev_delta_1/2/3` (steps), `prev_hfr_delta_1/2/3`	÷ max_step
Direction info (3)	`dir_changed_recently` (bool), `steps_since_flip_norm`, `consec_improvements_norm`	—
Rig params (3)	`backlash_norm`, `cfz_norm`, `filter_offset_norm`	÷ max_step
Sky (3)	`background_norm`, `saturation_frac`, `humidity_norm`	z-score / 0-1

v1: LightGBM. Trained offline on `focus_offset_ds_v1`; provides the first operational predictor.

```
import lightgbm as lgb

params = {
    "objective":         "huber",
    "alpha":             0.90,       # Huber robustness quantile
    "metric":            ["huber", "mae"],
    "n_estimators":      1000,
    "learning_rate":     0.05,
    "max_depth":         6,
    "num_leaves":        31,
    "min_child_samples": 20,
    "subsample":         0.80,
    "colsample_bytree":  0.80,
    "reg_alpha":         0.10,
    "reg_lambda":        0.10,
    "n_jobs":            -1,
    "seed":              42,
}
model = lgb.LGBMRegressor(**params)
model.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    callbacks=[lgb.early_stopping(50), lgb.log_evaluation(100)],
)
```

Listing 10: LightGBM v1 hyperparameters

v1 MLP distillation for ONNX edge deployment. The LightGBM model is too large for fast edge inference. A compact MLP is trained to match its predictions (knowledge distillation), then exported to INT8 ONNX.

```
class FocusOffsetMLP(nn.Module):
    def __init__(self, n_features: int = 38):
        super().__init__()
```

```
        self.trunk = nn.Sequential(
            nn.Linear(n_features, 128), nn.GELU(),
            nn.Dropout(0.15),
            nn.Linear(128, 64),  nn.GELU(),
            nn.Dropout(0.10),
        )
        # Multi-task heads
        self.head_delta     = nn.Linear(64, 1)   # signed delta steps
        self.head_log_sigma = nn.Linear(64, 1)   # log uncertainty
        self.head_direction = nn.Linear(64, 3)   # logits: [inward,
    stop, outward]

     def forward(self, x):
        h = self.trunk(x)
        return (self.head_delta(h).squeeze(-1),
                self.head_log_sigma(h).squeeze(-1),
                self.head_direction(h))
```

Listing 11: Distilled MLP architecture

Multi-task loss.

$$\mathcal{L}_{\text{total}} = \underbrace{\lambda_1 \, \mathcal{L}_{\text{huber}}(\hat{\delta}, \delta)}_{\text{regression}} + \underbrace{\lambda_2 \, \mathcal{L}_{\text{CE}}(\hat{d}, d)}_{\text{direction}} + \underbrace{\lambda_3 \, \mathcal{L}_{\text{hetero}}(\hat{s}, \delta)}_{\text{uncertainty}} \tag{2}$$

with $\lambda_1 = 1.0$, $\lambda_2 = 0.30$, $\lambda_3 = 0.20$.

Balanced sampling. Autofocus runs cluster near the optimum. Without resampling, the model becomes accurate only for small offsets and fails when the focuser is far from focus.

```
import numpy as np
from torch.utils.data import WeightedRandomSampler

bins   = [0, 5, 15, 40, 120, np.inf]   # step magnitude buckets
labels = np.digitize(np.abs(y_train), bins)
counts = np.bincount(labels)
weights = 1.0 / counts[labels]
sampler = WeightedRandomSampler(weights, num_samples=len(weights),
   replacement=True)
```

Listing 12: Balanced sampler over offset magnitude buckets

v2: MLP with quality embedding fusion. In v2, the penultimate layer of the FQE (576-dim, post-pool, pre-head) is concatenated with the tabular features to give the FOP additional perceptual signal. The trunk expands to `Linear(38+576, 256)` as the first layer. All other components remain identical.

7.6.3 Model 3: Focus Drift Forecaster (FDF)

Purpose. Predict focus drift as a function of time and temperature to trigger proactive refocus before sub quality degrades, rather than reacting after the fact.

v1: Kalman filter with linear temperature model. State vector: $\mathbf{x} = [p_{\text{focus}}, \, \dot{p}]^\top$ (position and drift rate).

Matrix	Form	Description
$F = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix}$	Constant-velocity model	State transition over Δt minutes
$H = \begin{pmatrix} 1 & 0 \end{pmatrix}$	Position observable	We observe position only
$Q = \mathrm{diag}(\sigma_p^2,\ \sigma_{\dot{p}}^2)$	$\sigma_p = 2\,\text{steps}$, $\sigma_{\dot{p}} = 0.5\,\text{steps/min}$	Process noise
$R = [\sigma_z^2]$	$\sigma_z = 3\,\text{steps}$	Measurement noise

Predict step: $\hat{\mathbf{x}}_{k|k-1} = F\hat{\mathbf{x}}_{k-1|k-1}, \quad P_{k|k-1} = FP_{k-1|k-1}F^\top + Q$

Update step (on receipt of a new autofocus position z_k):

$$K_k = P_{k|k-1}H^\top(HP_{k|k-1}H^\top + R)^{-1}, \quad \hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + K_k(z_k - H\hat{\mathbf{x}}_{k|k-1})$$

Temperature coefficient α (steps/°C): Estimated from consecutive focus runs. After each successful run:

$$\alpha_{\text{new}} = \frac{p_k - p_{k-1}}{T_k - T_{k-1}}, \qquad \alpha_{\text{rig}} \leftarrow 0.8\,\alpha_{\text{rig}} + 0.2\,\alpha_{\text{new}}$$

Stored in `focuser_profiles.temp_coefficient_steps_per_c`.

Drift forecast at time t:

$$p_{\text{forecast}}(t) = p_{\text{now}} + \alpha_{\text{rig}}\big(T_{\text{forecast}}(t) - T_{\text{now}}\big) + \dot{p}\,(t - t_{\text{now}})$$

Refocus recommendation triggered when:

$$|p_{\text{forecast}}(t) - p_{\text{now}}| > 0.5 \times \text{CFZ}$$

where CFZ (Critical Focus Zone, in steps) is stored per-rig.

Uncertainty propagation. Using the posterior covariance from the Kalman filter:

$$\sigma^2_{\text{forecast}}(t) = HF^n P_{k|k}(F^n)^\top H^\top + \alpha_{\text{rig}}^2\,\sigma^2_{T,\text{forecast}}$$

Output: `uncertainty_steps` $= \sqrt{\sigma^2_{\text{forecast}}}$.

7.6.4 Model 4: Cloud / Obstruction Classifier (COC)

Purpose. Detect partial or full cloud cover and photometric obstructions to prevent false bad-focus decisions and trigger safe parking or pausing.

Architecture. MobileNetV3-Small binary classifier.

- Replace final Linear(1024, 1000) with Linear(1024, 1) + Sigmoid
- Input: 224×224, 1-channel grayscale (same preprocessing as FQE)
- Temperature scaling: $\hat{p}_{\text{cal}} = \sigma(\text{logit}/T)$, T learned by minimizing NLL on held-out validation set (LBFGS, 100 steps)

Loss. Focal loss to handle class imbalance (clear nights are more common):

$$\mathcal{L}_{\text{focal}} = -\alpha(1-\hat{p})^\gamma \log\hat{p} - (1-\alpha)\hat{p}^\gamma \log(1-\hat{p}), \quad \alpha = 0.25,\ \gamma = 2.0$$

Training strategy.

- Hard negative mining: include near-clear, high-transparency frames as hard negatives (borderline cases near the cloud front)
- Augmentation: rotation (0/90/180/270°), horizontal flip, brightness (±30%), contrast (±20%)
- 80/20 stratified train/val by condition type (clear, thin cirrus, cumulus, overcast)

Inference hysteresis. Do not trigger a cloud pause on a single high-confidence frame. Apply hysteresis:

```
const CLOUD_THRESHOLD:  f32 = 0.80;
const CLEAR_THRESHOLD:  f32 = 0.30;
const CONSEC_REQUIRED:  u8  = 3;      // frames before acting

pub fn update_cloud_state(
    state: &mut CloudState, prob: f32
) -> CloudEvent {
    match state.status {
        CloudStatus::Clear => {
            if prob > CLOUD_THRESHOLD {
                state.consec_above += 1;
                if state.consec_above >= CONSEC_REQUIRED {
                    state.status = CloudStatus::Cloudy;
                    return CloudEvent::CloudDetected;
                }
            } else { state.consec_above = 0; }
        }
        CloudStatus::Cloudy => {
            if prob < CLEAR_THRESHOLD {
                state.consec_below += 1;
                if state.consec_below >= CONSEC_REQUIRED {
                    state.status = CloudStatus::Clear;
                    return CloudEvent::ClearedUp;
                }
            } else { state.consec_below = 0; }
        }
    }
    CloudEvent::None
}
```

Listing 13: Cloud detection hysteresis in Rust

7.6.5 Model 5: Trailing / Elongation Detector (TED)

Purpose. Detect guide failures, wind buffeting, and mount tracking errors that elongate stars, enabling the system to shorten exposure or recalibrate the guider.

Classical component (primary).

1. Source extraction: SEP (C backend), minimum area 5 px, detection threshold 3σ
2. Ellipse fitting: second-moment tensor from SEP output gives semi-axes a, b
3. Per-star ellipticity: $e_i = 1 - b_i/a_i$ (0 = circular, 1 = completely elongated)
4. Frame ellipticity: $e_{\text{frame}} = \tilde{\mu}_{0.2}(\{e_i\})$ (20% trimmed mean)
5. Trigger: trailing detected if $e_{\text{frame}} > 0.15$ with $N_{\text{stars}} \geq 10$
6. Direction: position angle of mean semi-major axis (indicates RA vs Dec drift)

CNN component (ambiguous cases). When $0.10 < e_{\text{frame}} < 0.20$ or $N_{\text{stars}} < 10$, a small CNN evaluates individual star cutout patches:

```
class TrailingCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
```

```
            nn.Conv2d(1, 32, 3, padding=1), nn.BatchNorm2d(32), nn.
    GELU(),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.
    GELU(),
            nn.MaxPool2d(2),                                    # 16x16
            nn.Conv2d(64, 128, 3, padding=1), nn.GELU(),
            nn.AdaptiveAvgPool2d(1),                            # 128-dim
        )
        self.head = nn.Sequential(
            nn.Linear(128, 64), nn.GELU(),
            nn.Linear(64, 2),   # [trailing_logit, severity_score]
        )
     def forward(self, x):   # x: (B, 1, 32, 32) star cutouts
        return self.head(self.conv(x).flatten(1))
```

Listing 14: CNN architecture for ambiguous trailing cases

Inference on up to 20 star patches per frame, majority vote for final decision.

7.6.6 Model 6: Background / SNR Estimator

Purpose. Estimate sky background, shot-noise floor, and per-pixel SNR for exposure and quality decisions.

Classical implementation (always active).

```
import sep, numpy as np

def estimate_background(data: np.ndarray) -> dict:
    bkg = sep.Background(data.astype(np.float32),
                         bw=64, bh=64, fw=3, fh=3)
    bkg_map  = bkg.back()             # 2-D interpolated map
    bkg_rms  = bkg.rms()              # noise map
    data_sub = data - bkg_map         # background-subtracted
    # SNR proxy: peak_ADU / sqrt(bkg_rms^2 + readnoise^2)
    snr_proxy = data_sub / np.sqrt(bkg_rms**2 + READNOISE**2)
    return {
        "background_adu":    float(np.median(bkg_map)),
        "background_sigma":  float(np.median(bkg_rms)),
        "snr_proxy_median":  float(np.median(snr_proxy[data_sub > 3*
    bkg_rms])),
        "saturation_fraction": float((data > 0.95 * SENSOR_SATURATION
    ).mean()),
     }
```

Listing 15: Background estimation via SEP (Python bindings)

Optional DnCNN-lite noise map (v2, GPU only). A 5-layer DnCNN variant trained on synthetic Poisson+Gaussian noise: $\text{Conv}(1, 64, 3) \to [\text{BN} + \text{ReLU} + \text{Conv}(64, 64, 3)] \times 3 \to \text{Conv}(64, 1, 3)$ with residual skip (noise map estimate, not denoised image). Only activated when a GPU is available; otherwise classical estimator is used.

7.6.7 Model 7: Exposure / Gain Recommender (LinUCB Contextual Bandit)

Purpose. Recommend optimal exposure time and gain for the current sky and target conditions, maximizing sub acceptance rate and SNR.

Context features (17-dim).

Feature	Encoding
Sky background ADU	$\log(\text{bkg}+1)$, z-score
Moon illumination fraction	$[0,1]$
Moon–target angular separation	$\cos(\theta_{\text{moon}})$
Filter (7-dim one-hot)	L, R, G, B, Ha, OIII, SII
Target type (4-dim one-hot)	galaxy, nebula, open cluster, globular
Seeing estimate (star FWHM)	z-score
Airmass	$[1,3]$ normalized
Guiding RMS (arc-sec)	z-score

Action space. Discrete exposure options: $\{10, 30, 60, 90, 120, 180, 300\}$ seconds.

LinUCB algorithm. For each arm a (exposure value), maintain $A_a \in \mathbb{R}^{d\times d}$ and $\mathbf{b}_a \in \mathbb{R}^d$:

$$\boldsymbol{\theta}_a = A_a^{-1}\mathbf{b}_a \tag{3}$$

$$\text{UCB}_a(\mathbf{x}) = \boldsymbol{\theta}_a^\top \mathbf{x} + \underbrace{\alpha\sqrt{\mathbf{x}^\top A_a^{-1}\mathbf{x}}}_{\text{exploration bonus}}, \quad \alpha = 1.0 \tag{4}$$

$$a^* = \arg\max_a \text{UCB}_a(\mathbf{x}) \tag{5}$$

Update on reward r: $A_a \leftarrow A_a + \mathbf{x}\mathbf{x}^\top$, $\mathbf{b}_a \leftarrow \mathbf{b}_a + r\mathbf{x}$.

Reward definition.

$$r = \underbrace{+1.0}_{\text{sub accepted}} + \underbrace{+0.5 \cdot \mathbb{1}[\text{SNR in target range}]}_{\text{quality bonus}} - \underbrace{0.5 \cdot \mathbb{1}[\text{sub rejected}]}_{\text{rejection penalty}} - \underbrace{0.3 \cdot \mathbb{1}[\text{saturation} > 5\%]}_{\text{saturation penalty}} - \underbrace{0.2 \cdot \mathbb{1}[\text{trailin}}_{\text{trailing}}$$

v2: Thompson Sampling. Maintain Bayesian posterior $\boldsymbol{\theta}_a \sim \mathcal{N}(\boldsymbol{\mu}_a, \sigma_a^2 A_a^{-1})$ where $\sigma_a^2 = 1$. At decision time, sample $\tilde{\boldsymbol{\theta}}_a$ from the posterior and select $a^* = \arg\max_a \tilde{\boldsymbol{\theta}}_a^\top \mathbf{x}$. Thompson Sampling has better finite-sample regret than LinUCB in high-noise environments.

7.6.8 Model 8: Filter Order Recommender (FOR)

Purpose. Recommend the next filter to switch to, balancing sky conditions, moon phase, remaining session time, and per-filter integration requirements.

Feature set.

Feature group	Features
Target	Type (4 one-hot), RA/Dec (sin/cos), airmass
Sky	Moon illumination, moon–target angular sep, sky background per filter
Session	Elapsed time, remaining planned time, current filter
Performance	Historical rejection rate per filter (past 30 runs), mean HFR per filter
Weather	Seeing estimate, transparency score, humidity
Integration	Accumulated exposure per filter vs target ratio

Architecture. LightGBM multi-class classifier:

```
params = {
    "objective":    "multiclass",
    "num_class":    7,          # L, R, G, B, Ha, OIII, SII
    "metric":       ["multi_logloss", "multi_error"],
```

```
    "n_estimators": 500,
    "learning_rate": 0.05,
    "max_depth":    5,
    "num_leaves":   21,
    "min_child_samples": 15,
    "subsample":    0.80,
    "colsample_bytree": 0.80,
}
```

Listing 16: Filter recommender training config

Explainability: SHAP values computed at inference time. Top-3 contributing features surfaced in the UI (*"Moon angle favors narrowband; Ha recommended"*).

7.6.9 Model 9: Tri-Modal Embedding (TME)

Purpose. Embed astrophysical objects into a shared 256-D latent space from any combination of: (a) imaging data, (b) photometric metadata, (c) light curves, (d) spectra. Enables cross-modal retrieval, target ranking, and anomaly detection.

Image encoder: ViT-Small/16 (or ConvNeXt-Tiny for lower compute).

Parameter	Value
Input	224×224, 3-channel or 1-channel grayscale
Patch size	$16 \times 16 \Rightarrow 196$ tokens + 1 [CLS]
Embedding dim	384
Attention heads	6
Transformer blocks	12
MLP ratio	4.0 (FFN hidden: 1536)
Projection head	Linear(384, 256) $\rightarrow$ L2 normalize

Metadata encoder.

```
class MetadataMLP(nn.Module):
    def __init__(self, n_meta: int):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_meta, 128), nn.LayerNorm(128), nn.GELU(),
            nn.Linear(128, 256),   nn.LayerNorm(256), nn.GELU(),
            nn.Linear(256, 256),
        )
    def forward(self, x):
        return nn.functional.normalize(self.net(x), dim=-1)
```

Listing 17: Metadata MLP encoder

Metadata features: RA/Dec (sin/cos encoding), filter, exposure, target type, airmass, moon illumination, site latitude/longitude.

Light curve encoder (optional modality). Architecture: InceptionTime (1D multi-scale convolution) or 4-block 1D Transformer.

Input	Time series, variable length, padded to 512 points
Encoding	Temporal Transformer: $d = 128$, 4 heads, 4 blocks
Aggregation	CLS token or mean pool
Projection	Linear(128, 256) $\rightarrow$ L2 normalize

Spectrum encoder (optional modality). 1D ResNet-18 variant: Conv1d$(1, 64, 7, s = 2) \to$ 4 residual blocks with channels $(64, 128, 256, 512) \to$ GlobalAvgPool $\to$ Linear$(512, 256) \to$ L2 normalize.

InfoNCE loss (CLIP-style). For a batch of N (image, metadata) pairs with embeddings $\mathbf{Z}^{\text{img}}, \mathbf{Z}^{\text{meta}} \in \mathbb{R}^{N \times 256}$:

$$S_{ij} = \frac{\mathbf{z}_i^{\text{img}} \cdot \mathbf{z}_j^{\text{meta}}}{\tau}, \quad \tau = 0.07$$

$$\mathcal{L}_{\text{img}} = -\frac{1}{N} \sum_i \log \frac{e^{S_{ii}}}{\sum_j e^{S_{ij}}}, \quad \mathcal{L}_{\text{meta}} = -\frac{1}{N} \sum_j \log \frac{e^{S_{jj}}}{\sum_i e^{S_{ij}}}$$

$$\mathcal{L}_{\text{InfoNCE}} = \frac{\mathcal{L}_{\text{img}} + \mathcal{L}_{\text{meta}}}{2}$$

For three modalities (img, meta, spectrum), the loss is averaged over all $\binom{3}{2} = 3$ pairs.

FAISS retrieval index.

```
import faiss, numpy as np

d = 256                              # embedding dimension
nlist = 256                          # Voronoi cells
M = 32                               # PQ sub-quantizers
nbits = 8                            # bits per sub-quantizer

quantizer = faiss.IndexFlatIP(d) # inner-product (L2-normalized =
    cosine)
index = faiss.IndexIVFPQ(quantizer, d, nlist, M, nbits)
index.train(embeddings_train)    # requires ~65k+ vectors
index.add(embeddings_train)
index.nprobe = 16                # cells to visit at query time
```

Listing 18: FAISS IVF-PQ index construction

Refresh cycle: nightly re-indexing as new sessions are synced to cloud.

7.6.10 Edge Model Filesystem Layout and Metadata

Listing 19: ONNX model directory structure

```
/models/
  focus_quality/
    1.0.0/  model.onnx  metadata.json  calibration.json  norm_params.json
  focus_offset/
    0.3.0/  model.onnx  metadata.json  norm_params.json
  focus_drift/
    0.1.0/  kalman_state.json  profile_registry.json
  cloud_classifier/
    1.2.0/  model.onnx  metadata.json  temperature.json
  trailing_detector/
    0.8.0/  model.onnx  metadata.json
```

Listing 20: metadata.json – full schema

```
{
  "name":    "focus_quality",
  "version": "1.0.0",
  "model_type":   "quality",
  "framework":    "pytorch-2.2␣->␣onnx-17",
  "quantized":    true,
```

```
  "quantization": "int8-qat",
  "input_schema": {
    "image":    { "shape": [1, 1, 256, 256], "dtype": "float32",
                  "norm": "percentile_clip_1_99" },
    "metadata": { "shape": [1, 4], "dtype": "float32",
                  "features": ["temp_norm","filter_id_norm",
                               "pos_norm","log_star_count_norm"] }
  },
  "output_schema": {
    "hfr_pred":    { "dtype": "float32", "units": "arcsec" },
    "log_sigma_sq":{ "dtype": "float32", "note": "aleatoric uncertainty" }
  },
  "trained_on_dataset":   "focus_quality_ds_v1_2026_01_14",
  "rig_agnostic":         true,
  "latency_p50_ms":       7.2,
  "latency_p95_ms":       11.8,
  "val_hfr_mae_arcsec":   0.14,
  "val_spearman_rho":     0.961
}
```

7.7 Training Pipeline Specification

7.7.1 Dataset Contracts

focus_quality_ds_v1. One row per focus sample.

Group	Columns
Identity	`focus_sample_id`, `focus_run_id`, `rig_id`, `timestamp`
Instrument	`filter_name`, `preview_exposure_ms`, `sensor_temp_c`, `ambient_temp_c`
Photometric	`star_count`, `background_adu`, `saturation_fraction`
Classical	`classical_hfr`, `classical_hfd`, `fit_quality`, `measurement_valid`
Image	`crop_path` (or binary tensor reference)
Target	`target_hfr` = `classical_hfr`

focus_offset_ds_v1. All quality columns plus:

Group	Columns
Run context	`run_best_fit_position`, `run_success`, `start_position`, `sample_index`
History	`prev_delta_steps_1/2/3`, `prev_hfr_delta_1/2/3`, `direction_changed_recently`
Rig params	`filter_offset_steps`, `temp_coefficient_steps_per_c`
Targets	`target_offset_steps`, `target_direction`, `target_abs_bucket`

7.7.2 Split Strategy

Important

Never use random row-level splitting. Multiple samples from the same run in both train and test sets constitutes severe data leakage. Split by *run* and *date*.

Oldest 70% of runs → train; next 15% → validation; newest 15% → test. Hold out one or more rigs entirely for a generalization evaluation set.

7.7.3 Preprocessing

Images (quality model): 256×256 grayscale center crop (or star-patch multi-crop). Convert to `float32`. Robust normalization via 1st–99th percentile clipping to $[0, 1]$. Store exact

normalization parameters for identical preprocessing on edge.

Tabular (offset model): Normalize temperature; log-transform background if needed; one-hot or learned embedding for filter; normalize focuser position by travel range.

Augmentations (quality only): Physically plausible: noise perturbation, mild blur, hot pixel simulation, brightness/contrast scaling. Never augmentations that change focus semantics.

7.7.4 Training Stages

Stage A: Extract. SQLite exports → canonical Parquet: `focus_quality_ds_v1.parquet`, `focus_offset_ds_v1.parquet`.

Stage B: Validate. Schema checks: non-null targets, plausible HFR ranges, positions inside bounds, unique (run_id, sample_index), image paths exist.

Stage C: Split. Deterministic run-level manifests with no leakage.

Stage D: Train Quality Model. Save checkpoints, metrics, calibration stats, latency benchmark.

Stage E: Generate Embeddings. Optionally cache penultimate-layer embeddings for offset model input.

Stage F: Train Offset Model. Engineered features ± quality embedding.

Stage G: Replay Evaluation. Simulate policy on historical runs; must meet promotion thresholds.

Stage H: Export. ONNX, metadata bundle, normalization params, calibration metrics.

Stage I: Shadow Deployment. Edge runs candidate model in shadow mode; logs `focus_predictions`.

7.7.5 ML Project Layout

```
focus_ml/
  datasets/
    build_focus_quality_dataset.py
    build_focus_offset_dataset.py
    split_registry.py
  preprocess/
    crops.py
    normalize.py
    feature_builder.py
  models/
    focus_quality_cnn.py
    focus_offset_mlp.py
    focus_offset_lgbm.py
  train/
    train_focus_quality.py
    train_focus_offset.py
    calibrate_uncertainty.py
  eval/
    eval_quality.py
    eval_offset.py
    replay_offset_policy.py
  export/
    export_quality_onnx.py
    export_offset_onnx.py
    write_model_metadata.py
```

Listing 21: Python ML project structure

7.8 Focus Controller Algorithm

7.8.1 Bounded AI Autofocus (Production MVP)

Phase 1 — Baseline Capture
Acquire preview at current position. Compute classical HFR and AI quality score. Verify sufficient stars. Abort to classical fallback if observation quality is poor.

Phase 2 — Offset Prediction
Run `focus_offset` model; obtain `delta_steps` and `uncertainty_steps`.

Phase 3 — Predicted Move
Propose `delta_steps`; clamp through `safety_guard`; apply backlash compensation if direction changed; move focuser.

Phase 4 — Measure Improvement
Settle → acquire preview → recompute metrics → compare against prior state.

Phase 5 — Local Refinement
If improved: continue with smaller bounded steps. If not improved: reverse or switch to bracket search.

Phase 6 — Convergence Decision
Stop when improvement $< \varepsilon$ over N steps, HFR plateaus, uncertainty too high, retry budget exhausted, or travel budget exhausted.

Phase 7 — Classical Fallback
Run classical bracket + parabola fit around predicted optimum region.

7.8.2 Classical Fallback

Sample $\{-60, -30, 0, +30, +60\}$ steps around initial position. Fit parabola or robust V-curve. Choose estimated optimum; confirm with 1–2 local refinement points. Bracket width is adaptive: high uncertainty ⇒ wider bracket; low uncertainty ⇒ narrow refinement.

7.9 Focus Configuration

Listing 22: section of achernar.toml

```
[focus]
enabled       = true
autonomy_mode = "supervised_auto"
roi           = "center"
preview_exposure_ms = 1000
min_star_count      = 20
settle_ms           = 500

[focus.safety]
min_position             = 1000
max_position             = 32000
max_step_single          = 80
max_total_travel_per_run = 1200
min_confidence_to_act    = 0.70
max_failed_improvements  = 3
max_uncertainty_steps    = 20.0

[focus.backlash]
enabled                      = true
backlash_steps               = 18
preferred_approach_direction = "inward"
```

```
[focus.models]
quality_model = "./models/focus_quality/1.0.0/model.onnx"
offset_model  = "./models/focus_offset/0.3.0/model.onnx"
drift_model   = "./models/focus_drift/0.1.0/model.onnx"

[focus.classical]
fallback_search_radius = 80
fallback_step          = 20
fit_method             = "parabola"
```

7.10 Evaluation & Promotion Criteria

7.10.1 Model Metrics

Model	Metric	Notes
Quality	MAE on HFR	Primary regression metric
	Spearman ρ with classical HFR	Rank correlation
	ECE (calibration error)	Uncertainty quality
	Inference latency p95	Edge budget
Offset	MAE in steps	Primary
	Sign accuracy	Direction trust
	% $\lvert\text{err}\rvert \leq 5$, 10, 20 steps	Operational bands
	Uncertainty interval coverage	Calibration
Drift	MAE in drift steps	Primary
	Precision/recall for refocus-needed-in-20min	Scheduling

7.10.2 Operational Control Metrics

- Focus convergence success rate
- Median time to focus; median number of focuser moves
- Final HFR improvement ratio vs classical baseline
- Fallback rate; clamped-action rate; low-confidence abort rate
- Number of bad subs avoided by proactive refocus
- User override rate (proxy for system trust)

7.10.3 Testing Stages

1. **Offline model validation:** Evaluate on recorded autofocus data.
2. **Offline policy replay:** Simulate candidate policy on historical runs.
3. **Shadow mode:** AI proposes, classical executes; differences logged only.
4. **Bounded autonomy in lab:** Real hardware, controlled conditions, no unattended runs.
5. **Limited field rollout:** Selected rigs, supervised mode, full logs, fast rollback.

Important

Shadow mode is **non-negotiable** before field rollout. A model that passes offline replay but behaves incorrectly on live hardware in shadow mode must not graduate to bounded autonomy.

7.11 Autonomy Maturity Ladder & Implementation Roadmap

Level	Description	Deliverable
L0 — Deterministic Baseline	Classical autofocus, classical exposure rules, no autonomous ML actuation	Fully logged deterministic autofocus
L1 — AI Perception	Focus quality, cloud, trailing models; drift estimator; advisory only	Live focus quality score + drift alerts
L2 — AI Proposal	Offset predictor; AI proposes, user or classical approves	AI-initialized autofocus
L3 — Bounded Autonomy	AI moves focuser inside strict safety limits; automatic fallback; shadow-tested first	Production bounded AI autofocus
L4 — Adaptive Personalization	Per-rig thermal drift, per-filter offsets, contextual bandits	Self-calibrating telescope profiles
L5 — Autonomous Observatory	Coordinated focus/exposure/scheduling/weather response	Self-driving observatory

7.11.1 Phase Build Order

Phase 1 — Control Foundation: Build all new schema tables; `focus_agent` state machine; `safety_guard`; `action_arbiter`; classical autofocus; UI status surface.

Phase 2 — Data Collection: Autofocus sample capture pipeline; dataset exporter; training feature builder.

Phase 3 — Focus Quality Model: Train quality estimator; ONNX export; edge integration; live monitoring.

Phase 4 — Focus Offset Predictor: Tabular model on focus run data; integrate as bounded-move initialization.

Phase 5 — Drift Forecasting: Drift dataset; forecast service; proactive refocus trigger.

Phase 6 — Shadow Policy + Bounded Auto: Replay evaluator; shadow mode; confidence gating; limited bounded autonomy.

7.11.2 Extension Pattern to Other Subsystems

Subsystem	Key Observations	Safety Constraints	Model
Exposure controller	Sky background, saturation, guiding RMS, moon, filter	Max saturation, exposure cap, no change mid-exposure	Contextual bandit
Filter controller	Target type, moon state, remaining time, weather	No interrupt critical block, respect locked plans	GBTs
Mount / scheduling	Target ranking, refocus timing, meridian, weather	Hard safety rails, deterministic motion	Planner / ILP

8 Subsystem Control Architecture: Filter Wheel, Mount, and Shared Perception

The focuser is the first subsystem to receive closed-loop AI control, but the control architecture is explicitly designed to generalise. This section specifies the vision perception bus shared across all subsystems, and the concrete models and safety contracts for the filter wheel and mount controllers.

8.1 Shared Perception Bus

The most important architectural rule for multi-subsystem control is this:

Important

No subsystem controller ever touches a raw pixel. Every controller reads from a single `SharedObservation` struct that the vision layer publishes at 1–5 Hz. The vision models are the *only* things that process frames.

This means the vision layer (FQE, COC, TED, Background Estimator) is not a focuser-specific subsystem. It is a shared infrastructure service, analogous to a sensor bus in an autonomous vehicle. Each control subsystem subscribes to the fields it cares about.

8.1.1 SharedObservation Struct

```
/// Published by the vision layer at 1-5 Hz.
/// All subsystem controllers consume this; none read raw frames.
pub struct SharedObservation {
    pub timestamp: Instant,

    // --- From FocusQualityEstimator ---
    pub predicted_hfr:          f32,
    pub sharpness_score:        f32,
    pub quality_uncertainty:    f32,
    /// Star FWHM as a seeing proxy (arcsec)
    pub seeing_estimate_arcsec: f32,
    /// Number of valid stars detected in the frame
    pub star_count:             u32,

    // --- From CloudObstructionClassifier ---
    pub cloud_probability:      f32,
    pub cloud_event:            CloudEvent,  // None | CloudDetected |
    ClearedUp

    // --- From TrailingElongationDetector ---
    pub trailing_severity:      f32,         // 0.0 (none) -> 1.0 (
   severe)
    /// Position angle of mean trailing axis (degrees); None if no
   trailing
    pub trailing_direction_pa: Option<f32>,
    /// Which axis is responsible: RA, Dec, or ambiguous
    pub trailing_axis:          Option<TrailingAxis>,

    // --- From BackgroundEstimator ---
    pub sky_background_adu:     f32,
    pub background_sigma:       f32,
    pub snr_proxy_median:       f32,
    pub saturation_fraction:    f32,

    // --- From PlateSolver (lower cadence, ~every solve) ---
    pub pointing_error_arcmin: Option<f32>,
    pub wcs_residual_arcsec:    Option<f32>,
    pub last_solve_confidence: Option<f32>,
    pub last_solve_at:          Option<Instant>,

    // --- Derived composite (computed from above) ---
```

```
    pub frame_quality_score:   f32,  // [0,1]; gates exposure
   acceptance
    pub observation_valid:     bool, // false if stars < min or frame
    corrupt
}

pub enum CloudEvent  { None, CloudDetected, ClearedUp }
pub enum TrailingAxis { RA, Dec, Ambiguous }
```

Listing 23: Shared observation struct published by the vision layer

8.1.2 Vision Layer to Subsystem Feed Table

Vision Model	Focus Agent	Filter Controller	Mount Controller
FQE (HFR, sharpness, seeing, star count)	Primary state estimation; bracket width; convergence gate	Filter-switch quality check; post-switch HFR verify	Star count drop → resolve trigger; seeing → guider aggressiveness
COC (cloud probability)	Pause run; invalidate drift forecast sample	Stay narrowband in thin cloud; pause in overcast	Park mount; halt sequence safely
TED (trailing severity, axis, PA)	Abort run if severe (mechanical vibration corrupt)	Switch to shorter exposures; shorter subs reduce trail impact	Tune RA/Dec guider aggressiveness independently; identify bad axis
Background Estimator (sky ADU, SNR, saturation)	Reject focus sample if sky saturated; validate star metric	Per-filter SNR scoring; primary bandit context feature	—
Seeing estimate (from FQE star FWHM)	Widen bracket in poor seeing; increase uncertainty threshold	Prefer narrowband in poor seeing (sky emission dominates less)	Lower guider aggressiveness in turbulent seeing
Plate Solver (WCS, pointing error)	—	—	Pointing model update; proactive resolve scheduler

8.1.3 Vision Loop Runtime

```
pub async fn run_vision_loop(
    ctx:  &VisionContext,
    bus:  Arc<Mutex<SharedObservation>>,
) {
    loop {
        let frame = ctx.camera.grab_preview().await?;

        // All vision models run on the same frame in parallel
        let (quality, cloud, trailing, background) = tokio::join!(
            ctx.fqe.infer(&frame),
            ctx.coc.infer(&frame),
            ctx.ted.infer(&frame),
            ctx.bge.estimate(&frame),
        );

        let obs = SharedObservation {
```

```
            timestamp:              Instant::now(),
            predicted_hfr:          quality.predicted_hfr,
            sharpness_score:        quality.sharpness_score,
            quality_uncertainty:    quality.uncertainty,
            seeing_estimate_arcsec: quality.seeing_estimate,
            star_count:             quality.star_count,
            cloud_probability:      cloud.probability,
            cloud_event:            cloud.event,
            trailing_severity:      trailing.severity,
            trailing_direction_pa:  trailing.direction_pa,
            trailing_axis:          trailing.axis,
            sky_background_adu:     background.background_adu,
            background_sigma:       background.background_sigma,
            snr_proxy_median:       background.snr_proxy_median,
            saturation_fraction:    background.saturation_fraction,
            frame_quality_score:    compute_composite_quality(&quality
    , &cloud, &trailing),
            observation_valid:      quality.star_count >= ctx.cfg.
    min_star_count
                                    && !frame.is_corrupt,
            // plate solver fields updated separately on solve events
            ..bus.lock().await.clone()
        };

        *bus.lock().await = obs;

        // Emit telemetry event for UI
        ctx.telemetry.emit(WsEvent::VisionUpdate(&obs)).await;

        tokio::time::sleep(ctx.cfg.vision_interval).await;
    }
}
```

Listing 24: Vision loop publishing to subsystem bus

8.2 Filter Wheel Subsystem

The filter wheel controller solves three nested problems: (1) **pre-session planning** (what is the ideal filter sequence for tonight?), (2) **real-time switching** (given current conditions, switch now or continue?), and (3) **post-switch handshake** (trigger focus offset correction before resuming imaging).

8.2.1 Filter-Focus Handshake Protocol

Every filter switch must coordinate with the focus agent. This is non-negotiable: a filter change without a corresponding focus offset correction produces out-of-focus subs.

```
pub async fn execute_filter_switch(
    ctx:       &mut FilterContext,
    focus_ctx: &mut FocusContext,
    target:    FilterName,
) -> Result<(), FilterError> {
    // 1. Assert ownership of both filter wheel and focuser
    ctx.arbiter.request(ControlDomain::FilterWheel, Priority::High)?;
    ctx.arbiter.request(ControlDomain::Focus,       Priority::High)?;

    // 2. Command wheel to rotate
```

```
    ctx.wheel.switch_to(target).await?;

    // 3. Settle (mechanical vibration)
    tokio::time::sleep(ctx.cfg.settle_ms).await;

    // 4. Apply stored per-filter focus offset
    let offset = ctx.profile.filter_offset(target);
    if offset != 0 {
        let action = FocusAction::RelativeMove { delta_steps: offset
   };
        let validated = focus_ctx.safety_guard.validate(action)?;
        focus_ctx.focuser.move_relative(validated.delta_steps).await
   ?;
        tokio::time::sleep(focus_ctx.cfg.settle_ms).await;
    }

    // 5. Optional: run short verification autofocus if filter offset
    is uncertain
    if ctx.profile.filter_offset_source(target) != OffsetSource::
   Measured {
        focus_ctx.run_short_verification_focus().await?;
    }

    // 6. Release ownership
    ctx.arbiter.release(ControlDomain::Focus);
    ctx.arbiter.release(ControlDomain::FilterWheel);

    ctx.log_switch(target).await?;
    Ok(())
}
```

Listing 25: Filter-focus handshake on every wheel switch

8.2.2 Model FW-1: Per-Filter Sky Scorer (Classical)

Before the bandit can choose a filter, it needs the expected SNR potential for each filter under current sky conditions. This is derived classically from short preview exposures (3–5 s each):

$$\text{SNR_potential}(f) = \frac{F_{\text{target}}(f) \cdot t}{\sqrt{F_{\text{target}}(f) \cdot t + \text{BKG_ADU}(f) \cdot t + N_{\text{pix}}\left(\sigma^2_{\text{dark}} t + \sigma^2_{\text{read}}\right)}} \tag{6}$$

where $F_{\text{target}}(f)$ is the estimated target flux in filter f (from catalog or prior sessions), $\text{BKG_ADU}(f)$ is the measured sky background per filter from the background estimator, and N_{pix} is the source aperture area in pixels.

The ratio SNR_potential(Ha)/SNR_potential(L) is the primary signal that makes the bandit prefer narrowband under moonlit conditions.

8.2.3 Model FW-2: Filter Sequencer (LinUCB Contextual Bandit)

Selects the next filter to image in, accounting for conditions, session progress, and target requirements.

Context vector (22 features).

Group	Features	Dim
Sky (6)	Moon illumination fraction, $\cos(\theta_{\text{moon-target}})$, log(sky_background_adu), seeing estimate, cloud_probability, SNR_potential ratio (Ha/L)	6
Session state (5)	Time remaining (normalized), integration ratio per filter vs target (Ha, OIII, L each as fraction complete), current filter (one-hot index)	5
Performance (8)	Rejection rate per filter (last 20 subs, 7 filters), mean HFR per filter (1 value)	8
Target (3)	Type one-hot (galaxy / nebula / cluster), estimated surface brightness	3

Action space. Switch to filter $f \in \{\text{L, R, G, B, Ha, OIII, SII}\}$, or "continue" (no switch). 8 discrete actions total.

LinUCB update. Identical to the exposure bandit (Section 7.6.7), with one $A_a, \mathbf{b}_a$ pair per filter action.

Reward.

$$r = +1.0\mathbb{1}[\text{sub accepted}] + 0.4 \cdot \frac{\text{SNR_actual}}{\text{SNR_target}(f)} - 0.5\mathbb{1}[\text{sub rejected}] - 0.3\mathbb{1}[\text{cloud_prob} > 0.50] - 0.2\mathbb{1}[\text{trailing_sev} \tag{7}$$

Vision feeds.

- COC cloud_probability: high cloud probability during broadband → bandit learns to prefer narrowband (narrowband less sensitive to sky transparency variation)
- TED trailing_severity: active trailing → bandit penalises long-exposure filters (L, Ha) and rewards shorter subs (R, G, B) until mount settles
- Background estimator: per-filter sky ADU is recomputed from short preview exposures; primary input to SNR_potential
- FQE star_count: if star count drops without a cloud event, pointing has drifted and filter switch should be deferred until re-solve

8.2.4 Model FW-3: Session Plan Optimizer (ILP, OR-Tools)

Pre-session integer linear program that produces the initial filter sequence and time allocation before the bandit takes over for real-time adaptation.

Decision variables. $x_{f,k} \in \{0, 1\}$: image with filter f in time slot k. $t_{f,k} \in \mathbb{R}_{\geq 0}$: integration seconds allocated to filter f in slot k.

Objective.

$$\max \sum_{f,k} \text{SNR_potential}(f, k) \cdot t_{f,k} \cdot x_{f,k} \tag{8}$$

Constraints.

$$\sum_f x_{f,k} = 1 \quad \forall k \qquad \text{(one filter per slot)} \tag{9}$$

$$\sum_k t_{f,k} \geq T_{\min}(f) \qquad \text{(minimum integration per filter)} \tag{10}$$

$$x_{f,k} = 0 \text{ if altitude}(k) < 20 \qquad \text{(horizon constraint)} \tag{11}$$

$$x_{f,k} + x_{f',k+1} \leq 1 + y_{f,f',k} \qquad \text{(switch penalty via auxiliary } y\text{)} \tag{12}$$

$$\sum_k y_{f,f',k} \cdot c_{\text{switch}} \leq C_{\text{switch,max}} \qquad \text{(total switch cost budget)} \tag{13}$$

Solved with OR-Tools CP-SAT solver. Typical solve time $< 2\,\text{s}$ for a 6-hour session. The ILP result seeds the bandit with a warm-start policy; the bandit then adapts to actual conditions throughout the night.

8.2.5 Filter Wheel Safety Configuration

```
pub struct FilterWheelSafetyConfig {
    /// Minimum mechanical settle time after rotation (ms)
    pub min_settle_ms:            u64,
    /// Maximum filter switches per hour (mechanical wear limit)
    pub max_switches_per_hour:    u8,
    /// Never command a switch during an active exposure
    pub forbidden_during_exposure: bool,
    /// Always trigger focus offset correction on switch
    pub require_focus_handshake:  bool,
    /// Run a short verification focus if offset source is 'manual'
    pub verify_if_offset_manual:  bool,
    /// Cloud probability above which switching is deferred
    pub defer_switch_cloud_thresh: f32,
}
```

Listing 26: Filter wheel safety config struct

8.3 Mount Subsystem

The mount controller is the most safety-critical subsystem in the entire system. A miscommunication here can slew into an obstacle, collide with a pier, or leave the mount unparked in deteriorating weather. All ML components in this subsystem operate in **advisory** or **parameter-tuning** roles only; raw slew commands always pass through a deterministic safety layer.

Important

The mount safety guard must enforce hard-coded altitude limits, meridian flip windows, and pier collision zones derived from the rig geometry profile. No ML model is permitted to bypass these. The mount controller maturity ladder starts at L2 (AI Proposal) for parameter tuning and never proceeds to L3 (Bounded Autonomy) for slew commands without explicit user configuration and hardware interlock confirmation.

8.3.1 Model M-1: Guiding Aggressiveness Tuner (LinUCB)

Adjusts the internal guider's RA and Dec aggressiveness parameters in real time. This is the mount subsystem's most direct beneficiary of the shared vision bus — the trailing detector output directly tells the model which axis to tune.

Context vector (13 features).

Group	Features	Dim
Vision (4)	seeing_estimate_arcsec, trailing_severity, trailing_axis (RA=0/Dec=1/Ambiguous=0.5), cloud_probability	4
Guider telemetry (6)	RMS_{RA} (last 10 corrections), RMS_{Dec} (last 10), mean correction magnitude, correction frequency, time since calibration (normalized), star mass SNR	6
Environment (3)	Wind speed estimate (if sensor available), airmass, hour angle (normalized)	3

Action space. Two independent bandits — one per guiding axis:

$$a_{\text{RA}},\ a_{\text{Dec}} \in \{0.10,\ 0.30,\ 0.50,\ 0.70,\ 1.00,\ 1.30\}$$

6 arms each, so the combined action space is $6 \times 6 = 36$ pairs. In practice, the two bandits are decoupled and share only the context vector.

Reward.

$$r = -\underbrace{\text{RMS}_{\text{guide}}}_{\text{primary}} - 0.5 \cdot \underbrace{\text{trailing_severity}}_{\text{from TED}} + 0.3 \cdot \underbrace{\mathbb{1}[\text{sub accepted}]}_{\text{downstream quality}} - 0.4 \cdot \underbrace{\mathbb{1}[\text{oscillating}]}_{\text{over-correction detect}} \tag{14}$$

Oscillation detection: guider is oscillating if the sign of consecutive corrections alternates for > 5 samples on the same axis (indicating over-correction; aggressiveness too high for current seeing).

Key vision interactions.

- `trailing_axis`: if TED identifies RA-axis trailing, the RA bandit receives the reward signal and adjusts aggressiveness; Dec bandit is held constant
- `seeing_estimate`: high seeing variance → lower aggressiveness (chasing atmospheric noise makes guiding worse); seeing-adaptive aggressiveness is the primary learned behaviour
- `cloud_probability`: if cloud is rising, the reward signal becomes noisy (sub acceptance rate drops for non-guiding reasons); bandit exploration is frozen during cloud events to avoid spurious updates

8.3.2 Model M-2: Dither Magnitude Policy (Regression MLP)

Dithering between exposures removes fixed-pattern noise, hot pixels, and detector banding. Too little dithering fails to decorrelate noise; too much wastes time waiting for the guider to re-settle.

Features (10).

Feature	Source / Encoding
Guiding RMS (arcsec)	Guider telemetry, z-score
Image scale (arcsec/pixel)	Rig profile, fixed
Target type (4 one-hot)	Galaxy / nebula / open cluster / globular
Filter (7 one-hot)	Current filter
Sub rejection rate (recent)	Fraction of last 20 subs rejected
Session hot-pixel count	Dark calibration, normalized

Output.

- `dither_magnitude_pixels`: continuous, clamped to $[1, 15]$
- `settle_threshold_arcsec`: guider RMS below which to declare settled, clamped to $[0.3, 2.0]$

Architecture.

```
class DitherPolicyMLP(nn.Module):
    def __init__(self, n_features: int = 10):
        super().__init__()
        self.trunk = nn.Sequential(
            nn.Linear(n_features, 64), nn.GELU(),
            nn.Linear(64, 32),         nn.GELU(),
        )
        self.head_magnitude = nn.Sequential(
            nn.Linear(32, 1), nn.Sigmoid())  # scaled to [1,15]
        self.head_settle    = nn.Sequential(
            nn.Linear(32, 1), nn.Sigmoid())  # scaled to [0.3,2.0]

    def forward(self, x):
        h = self.trunk(x)
        mag     = self.head_magnitude(h) * 14.0 + 1.0
        settle  = self.head_settle(h)    * 1.7  + 0.3
        return mag.squeeze(-1), settle.squeeze(-1)
```

Listing 27: Dither policy MLP

Labels. Generated from historical sessions by correlating dither size with downstream sub acceptance rate and stacked image noise floor (measured post-session). This is a weakly supervised problem: the optimal dither is not directly observed, only its downstream consequence.

8.3.3 Model M-3: Plate Solve Trigger Classifier (LightGBM)

Proactively triggers a plate solve before pointing error accumulates enough to affect sub quality or cause the target to drift out of frame.

Feature set (11 features).

Feature	Source
Minutes since last solve	State clock
Temperature delta since last solve (°C)	Environmental sensor
Slew distance since last solve (degrees)	Mount telemetry
Meridian flip occurred since last solve (bool)	Mount state
Mount model RMS (arcsec)	Mount model
Last solve confidence	Plate solver output
Star count trend (slope over last 5 frames)	FQE star_count
Guiding RMS trend (slope over last 10 corrections)	Guider telemetry
Airmass	From RA/Dec + site
Hour angle (normalized)	Mount telemetry
Time since power-on (normalized)	Daemon uptime

Output. Binary: `solve_now` vs `wait`.

LightGBM config.

```
params = {
    "objective":          "binary",
    "metric":             ["binary_logloss", "auc"],
    "n_estimators":       300,
```

```
    "learning_rate":      0.05,
    "max_depth":          4,
    "num_leaves":         15,
    "min_child_samples":  10,
    "scale_pos_weight":   3.0,   # solves are rare; upsample
   positives
    "subsample":          0.80,
    "colsample_bytree":   0.80,
}
```

Listing 28: Plate solve trigger classifier config

Vision interactions.

- **FQE star_count trend**: a downward trend without a concurrent cloud event is the strongest single signal that pointing has drifted (target moving toward frame edge)
- **Guiding RMS trend**: rising RMS that is not explained by increased seeing (seeing held constant) suggests the mount model is deteriorating
- **COC cloud event**: a cloud detection followed by a clear event is a mandatory re-solve trigger (mount may have flexed during shutter-closed interval)

8.3.4 Model M-4: Pointing Model (Thin-Plate Spline)

Corrects for systematic position-dependent pointing errors arising from polar alignment error, orthogonality (cone) error, and mechanical flexure. This is a non-parametric spatial model, not a deep learning model.

Data collection. After each successful plate solve at sky position (α_i, δ_i), store the commanded position $(\alpha_i^{\text{cmd}}, \delta_i^{\text{cmd}})$ and the solved true position $(\alpha_i^{\text{true}}, \delta_i^{\text{true}})$:

$$\Delta\alpha_i = \alpha_i^{\text{true}} - \alpha_i^{\text{cmd}}, \quad \Delta\delta_i = \delta_i^{\text{true}} - \delta_i^{\text{cmd}}$$

Thin-Plate Spline fit. With N calibration points, fit two independent TPS interpolators for RA and Dec error:

$$f_{\text{RA}}(\alpha, \delta) = a_0 + a_1\alpha + a_2\delta + \sum_{i=1}^{N} w_i \phi\big(r_i(\alpha, \delta)\big)$$

where $\phi(r) = r^2 \log r$ is the TPS radial basis function and $r_i = \|(\alpha, \delta) - (\alpha_i, \delta_i)\|$.

Regularisation parameter $\lambda = 1 \times 10^{-4}$ prevents overfitting on sparse calibration data. Minimum 5 calibration points required; 20+ recommended for reliable sky-wide correction.

Application at slew time.

```
pub fn correct_pointing(
    tps:  &ThinPlateSpline,
    ra:   f64,   // commanded RA (degrees)
    dec:  f64,   // commanded Dec (degrees)
) -> (f64, f64) {
    let d_ra  = tps.ra_model.predict(ra, dec);
    let d_dec = tps.dec_model.predict(ra, dec);
    // Clamp correction to prevent pathological extrapolation
    let d_ra  = d_ra.clamp(-MAX_CORRECTION_DEG, MAX_CORRECTION_DEG);
    let d_dec = d_dec.clamp(-MAX_CORRECTION_DEG, MAX_CORRECTION_DEG);
    (ra - d_ra, dec - d_dec)
}
```

Listing 29: Pointing correction applied before slew

The model is rebuilt nightly from accumulated solve data and persisted in `device_profiles` as a serialized spline coefficient set.

8.3.5 Mount Safety Configuration

```
pub struct MountSafetyConfig {
    /// Minimum altitude for any slew target (degrees)
    pub min_altitude_deg:          f32,
    /// Maximum slew speed (degrees/sec) for imaging slews
    pub max_slew_speed_deg_per_s:  f32,
    /// Hour angle limits (hours) east and west of meridian
    pub hour_angle_limit_east_h:   f32,
    pub hour_angle_limit_west_h:   f32,
    /// Park if cloud detected for this many consecutive minutes
    pub cloud_park_timeout_min:    u32,
    /// Hard altitude horizon model (polygon, per-rig)
    pub horizon_model:             HorizonPolygon,
    /// Require user confirmation for any slew >30 deg
    pub confirm_large_slews:       bool,
    /// Meridian flip behaviour
    pub auto_meridian_flip:        bool,
    pub flip_lead_time_min:        u32,
}
```

Listing 30: Mount safety config — never bypassed by ML

8.4 Schema Additions for Filter and Mount Subsystems

```
CREATE TABLE filter_switch_events (
    id              TEXT PRIMARY KEY,
    session_id      TEXT REFERENCES sessions(id),
    rig_id          TEXT NOT NULL REFERENCES rigs(id),
    timestamp       DATETIME NOT NULL,
    from_filter     TEXT NOT NULL,
    to_filter       TEXT NOT NULL,
    trigger_type    TEXT NOT NULL
        CHECK (trigger_type IN (
            'manual','bandit','ilp_plan','quality_drop',
            'cloud_event','session_start')),
    settle_ms_actual    INTEGER,
    focus_offset_applied INTEGER,
    focus_handshake_ok  INTEGER NOT NULL DEFAULT 0,
    sky_background_before REAL,
    sky_background_after  REAL,
    bandit_ucb_score     REAL,
    bandit_arm_index     INTEGER,
    reward_observed      REAL
);
```

Listing 31: Filter wheel run log

```
CREATE TABLE guider_state_log (
    id              TEXT PRIMARY KEY,
    session_id      TEXT REFERENCES sessions(id),
    timestamp       DATETIME NOT NULL,
    rms_ra_arcsec   REAL,
    rms_dec_arcsec  REAL,
    aggressiveness_ra  REAL,
    aggressiveness_dec REAL,
    trailing_severity  REAL,
```

```
    trailing_axis      TEXT CHECK (trailing_axis IN ('RA','Dec','Ambiguous','
    None')),
    seeing_estimate    REAL,
    airmass            REAL,
    bandit_action_ra   REAL,
    bandit_action_dec  REAL,
    reward_observed    REAL
);

CREATE TABLE plate_solve_events (
    id              TEXT PRIMARY KEY,
    session_id      TEXT REFERENCES sessions(id),
    rig_id          TEXT NOT NULL REFERENCES rigs(id),
    timestamp       DATETIME NOT NULL,
    trigger_type    TEXT NOT NULL
        CHECK (trigger_type IN (
            'startup','post_slew','post_flip','proactive',
            'star_count_drop','manual')),
    commanded_ra_deg  REAL,
    commanded_dec_deg REAL,
    solved_ra_deg     REAL,
    solved_dec_deg    REAL,
    error_arcmin      REAL,
    confidence        REAL,
    solve_time_ms     INTEGER,
    success           INTEGER NOT NULL DEFAULT 0,
    classifier_score  REAL    -- M-3 model output that triggered this solve
);

CREATE TABLE pointing_model_snapshots (
    id              TEXT PRIMARY KEY,
    rig_id          TEXT NOT NULL REFERENCES rigs(id),
    created_at      DATETIME NOT NULL,
    calibration_points INTEGER NOT NULL,
    rms_arcsec      REAL,
    max_error_arcsec REAL,
    tps_coefficients_json TEXT NOT NULL  -- serialized spline
);
```

Listing 32: Guider state log for bandit training

8.5 Subsystem Model Summary

Model	Subsystem	Type	Architecture	Where	Vision Input
FQE	Focus	Regression	MobileNetV3-Small + MLP	Edge	Self
FOP	Focus	Regression	LightGBM → MLP	Edge	FQE output
FDF	Focus	Forecasting	Kalman + linear temp	Edge	None
COC	All	Classification	MobileNetV3-Small	Edge	Self
TED	All	Detection	Classical SEP + CNN	Edge	Self
BGE	All	Estimation	Classical SEP	Edge	Self
FW-1	Filter	SNR scoring	Classical equation	Edge	BGE output
FW-2	Filter	Bandit	LinUCB (7+1 arms)	Edge	COC, TED, BGE, FQE
FW-3	Filter	ILP planning	OR-Tools CP-SAT	Edge/offline	None (pre-session)
M-1	Mount	Bandit (2 axes)	LinUCB (6 arms × 2)	Edge	TED, FQE seeing
M-2	Mount	Regression	3-layer MLP	Edge	None (guider tel.)
M-3	Mount	Classification	LightGBM binary	Edge	FQE star_count, COC
M-4	Mount	Spatial model	Thin-Plate Spline	Edge/nightly	None (solve events)
Exp.	Imaging	Bandit	LinUCB (7 arms)	Edge	BGE, TED, FQE
FOR	Imaging	Classification	LightGBM multi-class	Edge	BGE, FQE seeing
TME	Cloud	Embedding	ViT + MLP + TF + 1D-ResNet	Cloud	None (trained offline)

9 Data Engineering & Labeling

9.1 Bootstrapped Labels from Classical Pipeline

- HFR/HFD, star count, background stats, elongation become targets or features
- Every autofocus run generates labeled training data automatically—the highest-value dataset in the system
- Quality: combine multiple classical metrics into a composite score

9.2 Programmatic Weak Labels (Snorkel-Style)

Rules for "good/bad sub," "cloudy," "oversaturated," "trailing." Label aggregation via majority vote or learned label model.

9.3 Public Archives

- Images: SDSS, PanSTARRS survey tiles
- Light Curves: ZTF/ASAS-SN/TESS

- Spectra: SDSS/LAMOST

9.4 Augmentations

- **Images:** Synthetic blur (PSF), noise injection (shot+read), gradients, dithers, hot pixels, cosmic rays
- **Light Curves:** Jitter, missingness masks, amplitude scaling, phase shifts
- **Spectra:** Resolution blur, noise, line shifts, dropout, continuum variations

10 Metrics

10.1 Edge Models

- **Focus Regressor:** MAE vs HFR; Spearman ρ; latency ms/frame; ECE
- **Cloud Classifier:** AUROC; precision at high recall; F1
- **Trailing Detector:** Precision/recall; false positive rate at threshold

10.2 Recommender Systems

Exposure/Gain: off-policy eval (IPS/DR); uplift vs baseline; user acceptance rate. Filter Choice: accuracy on held-out sessions; regret; user override rate.

10.3 System-Level Metrics

Session duration; repeat usage; % subs kept vs rejected; median SNR improvement; focus convergence success rate; proactive refocus success; user override rate.

11 Training/Serving Stack

11.1 Development Infrastructure

- Feature Store & Lineage: Parquet + DuckDB/Polars; schema versioning
- Versioning: DVC (data), MLflow (experiments), model registry
- Training: PyTorch Lightning; mixed precision (AMP); quantization-aware training (QAT) for edge
- Hyperparameter Tuning: Optuna or Ray Tune

11.2 Serving Infrastructure

- Edge: ONNX Runtime, INT8 quantized, OpenVINO/TensorRT optional
- Cloud: gRPC or FastAPI; TorchServe or TFServing
- Indexing: FAISS (IVF-PQ) with periodic refresh
- Caching: Redis with TTL

12 Edge vs Cloud: Who Runs What?

12.1 Edge (<50 ms Latency)

Focus quality/offset/drift models; cloud/trailing classifiers; Kalman drift; rules/bandit for settings; all classical stats; safety guard; action arbiter.

12.2 Cloud (Heavier Compute)

Target ranking/scheduling; tri-modal embedding training; anomaly detection at scale; model retraining with active learning; A/B testing; canary deployments; replay policy evaluation; federated learning aggregation.

13 Network Layer

- Local WiFi AP: hostapd, dnsmasq, iptables NAT, WPA3
- Internet: NetworkManager, mobile hotspot, connection priority
- VPN: WireGuard, Tailscale, dynamic DNS
- Remote Control: WebSocket command tunnel (WSS) via Django relay; WebRTC P2P video stream; TURN fallback

14 Achernar Web Platform (AWS/GCP)

14.1 Control & Data Planes

- Control Plane: Secure WebSockets (WSS); Django Channels + Redis; <100 ms latency
- Data Plane: WebRTC (UDP) for 720p/1080p H.264 stream; TURN server fallback

14.2 Backend Services

Django 4.2+, PostgreSQL 15+, Celery + Redis, S3/GCS + CDN, FAISS vector search.

14.3 AI/ML Cloud Services

Tri-modal embedding (ViT/ConvNeXt image, Temporal Transformer light curve, 1D ResNet spectra), 256-D InfoNCE shared space, LightGBM target ranking, Isolation Forest anomaly detection, Bayesian hierarchical bandits.

15 Pragmatic Cautions

1. **Never denoise raw subs with ML**—preserve raw science integrity
2. **Always keep classical fallback available**—never ship ML without it
3. **Make edge models tiny**—test latency on target hardware early
4. **Calibrate all probabilities**—uncalibrated confidence is worse than none
5. **Shadow mode before any bounded autonomy**—non-negotiable
6. **Do not start with RL on physical hardware**—build offline replay first
7. **Beware feedback loops**—periodically surface random samples, not only AI-curated data
8. **Test on representative rigs**—backyard setup $\neq$ desert observatory
9. **Explain failures loudly**—build trust through transparency
10. **Measure operational metrics**—convergence rate matters more than model accuracy

16 Updated R&D Roadmap

16.1 Weeks 1–2: Foundation

Classical metrics + logging. Focus quality model (MobileNetV3-Small) + ONNX INT8 export. Build all new focus schema tables. Unit tests for inference latency (<50 ms).

16.2 Weeks 3–4: Quality Control + Control Foundation

Cloud/obstruction classifier. Exposure bandit v0 (LinUCB). `focus_agent` state machine, `safety_guard`, `action_arbiter`. Classical autofocus with full logging.

16.3 Weeks 5–6: Embeddings, Ranking, and Offset Predictor

Bimodal embedding (image + metadata) with CLIP-style InfoNCE. Target ranking with OR-Tools. Focus offset predictor v1 (LightGBM). Integrate as bounded-move initialization.

16.4 Weeks 7–8: Multi-Modal, Feedback, and Shadow Mode

Light-curve encoder. Calibrated quality scoring. User feedback collection. Shadow mode for offset predictor. Replay evaluation and promotion criteria.

16.5 Weeks 9–10: Bounded Autonomy

Shadow-mode field testing. If promotion criteria met, enable bounded autonomy in `supervised_auto` for selected rigs. Monitor convergence rate, fallback rate, clamp rate.

A Technology Stack Summary

A.1 Core Daemon

Rust + C++ (FFI), tokio, Axum, OpenCV, libraw, ONNX Runtime, SQLite, FITS/TIFF media store.

A.2 Desktop Application

Tauri 2.0, React 18 + TypeScript, local persistence and daemon-backed UI.

A.3 Web Platform

Django 4.2+, Python 3.11+, React 18, Next.js/Vite, Tailwind CSS, PostgreSQL 15+, Celery + Redis, S3/GCS + CDN.

A.4 Key Libraries

- **Edge:** `onnxruntime-rs`, OpenCV, SEP/Photutils, Eigen (C++), torch (QAT), onnx-simplifier
- **Cloud/Training:** PyTorch Lightning, LightGBM/XGBoost, FAISS, OR-Tools, MLflow, DVC, Polars/DuckDB, Optuna, Ray, SHAP, LIME, Great Expectations

B Achernar vs. Traditional Approaches

Feature	Traditional	Achernar
Offline Operation	Limited/None	Full functionality
AI Guidance	None	Real-time, uncertainty-aware
Hardware Parameter Control	Manual only	Closed-loop AI with safety supervisor
Model Updates	N/A	Active learning from users
Explainability	Manual	Automated SHAP/attention
Multi-Modal Learning	Separate tools	Unified embedding space
Privacy	Data uploaded	Federated learning option
Deployment	Desktop-only	Edge + Cloud + PWA

C Future Research Directions

1. Self-Supervised Learning: leverage unlabeled astronomical data for pretraining
2. Transfer Learning: professional observatory models adapted to amateur equipment
3. Causal Inference: seeing conditions → image quality causal models
4. Multi-Agent RL: coordinate multiple telescopes for optimal survey coverage (simulation-first)
5. Neural Architecture Search: automated optimal model architectures for edge deployment
6. Adversarial Robustness: ensure robustness to corrupted inputs and sensor artifacts
7. Continual Learning: adapt to new equipment/conditions without catastrophic forgetting
8. Simulation Environment: full telescope + optics simulator for safe RL training

D Hardware Requirements

- Raspberry Pi 4/5 (4 GB+ RAM recommended)
- NVIDIA Jetson Nano (for GPU acceleration)
- Intel NUC (11th gen or later)
- Personal laptops (Windows, macOS, Linux)